

INFORME TÉCNICO — SOY PACHÓN MUNDIAL (@SOYPACHON)

Entregable de documentación final. Este documento permite a equipos de soporte preventivo, correctivo y evolutivo entender, mantener y evolucionar la aplicación de manera autónoma.

Versión del documento: 1.0 · Última actualización: ver sección 13.

TABLA DE CONTENIDOS

1. Resumen ejecutivo
2. Arquitectura de la aplicación
3. Base de datos
4. Módulos funcionales
5. APIs y servicios externos
6. Infraestructura y hosting
7. Guía de operación y administración
8. Comandos Artisan de referencia
9. Soporte preventivo
10. Soporte correctivo
11. Soporte evolutivo
12. Glosario
13. Historial de versiones
14. Cómo mantener este documento

1. RESUMEN EJECUTIVO

1.1 IDENTIFICACIÓN

ATRIBUTO	VALOR
Nombre	Soy Pachón Mundial
Marca pública	@SoyPachon
Propósito	Plataforma web de pronósticos para el Mundial FIFA 2026, dirigida a un grupo privado de participantes de pago. Los usuarios pronostican los marcadores de los 104 partidos del torneo, acumulan puntos según precisión y compiten por premios en efectivo.
URL de producción	https://soypachonmundial.online
Repositorio GitHub	https://github.com/efabianpq/soypachonmundial
Tipo de licencia	Proyecto privado

1.2 STACK TECNOLÓGICO COMPLETO

CAPA	TECNOLOGÍA	VERSIÓN
Lenguaje backend	PHP	8.3.30
Framework backend	Laravel	11.46
Base de datos producción	MySQL (Hostinger)	8.x
Base de datos local dev	MySQL 8.4.3 (Laragon)	8.4.3
Base de datos tests	SQLite in-memory	n/a
Composer	Composer	2.9.4
Frontend templating	Blade	(incluido en Laravel)
Frontend reactividad	Alpine.js	3
Frontend estilos	Tailwind CSS	3.4
Frontend builder	Vite	5
Frontend plugin	@tailwindcss/forms	latest
Generación PDF	barryvdh/laravel-dompdf	^3.1
Email producción	SMTP (Hostinger Mail)	n/a
Email desarrollo	Driver <code>log</code>	n/a
Node (solo build local)	Node.js	≥ 18
Hosting	Hostinger Premium Web Hosting	n/a
Dominio	soypachonmundial.online	n/a

1.3 ESTADO ACTUAL DEL PROYECTO

MVP Fase 1 — COMPLETADO (versión `v1.0.0` , tag `c774a8f`).

Funcionalidad implementada:

- Autenticación con códigos de invitación SPM-XXXX
- Fixture completo de 104 partidos (72 grupos + 32 eliminatoria)
- Pronósticos con autosave, bloqueo automático 5 min antes
- Motor de cálculo de puntos (regla 5/3/2/1/0 estricta same-side)
- Ranking público para usuarios activos con auditoría detallada
- Panel de administración completo
- Notificaciones por email (recordatorios 15 min antes del cierre)
- Exportación de auditoría en CSV y PDF
- Sección pública `/como-funciona` con calculadora de premios
- Design system aplicado completo

Fase 2 — En operación. El sitio está activo en producción.

Fase 3 — PENDIENTE. Integraciones diferidas para implementación futura:

- API-Football (resultados automáticos en tiempo real)
- Google OAuth (login con cuenta Google)
- CallMeBot WhatsApp / Twilio (notificaciones WhatsApp)
- Google reCAPTCHA v3 (anti-bot en registro)
- Pasarela de pagos (MercadoPago / PSE)

1.4 CONTACTOS DEL PROYECTO

ROL	PERSONA	CONTACTO
Administrador del proyecto	Fabian Pachón	WhatsApp +57 301 396 6515 · <code>efabianpq@gmail.com</code>
Email transaccional saliente	Sistema	<code>notificaciones@soypachonmundial.online</code>
Repositorio GitHub	<code>efabianpq</code>	<code>https://github.com/efabianpq/soypachonmundial</code>

2. ARQUITECTURA DE LA APLICACIÓN

2.1 DIAGRAMA DE ARQUITECTURA (ALTO NIVEL)

```

????????????????????????????????????????????????????????????????
? BROWSER (cliente) ?
? HTML5 + Blade-rendered HTML + Tailwind CSS + Alpine.js ?
? Vite-built assets (public/build/app-*.css / *.js) ?
????????????????????????????????????????????????????????????????
? HTTPS (puerto 443, SSL Let's Encrypt)
?
????????????????????????????????????????????????????????????????
? HOSTINGER PREMIUM · Apache + PHP-FPM ?
? DocumentRoot: ~/public_html ? symlink a ~/soypachonmundial/public?
????????????????????????????????????????????????????????????????
? HTTP request
?
????????????????????????????????????????????????????????????????
? LARAVEL 11 (~/soypachonmundial/public/index.php) ?
?
? ?????????????????????????????????????????????????????????????? ?
? ? 1. bootstrap/app.php ? Application instance ? ?
? ? 2. routes/web.php ? Router resuelve la URL ? ?
? ? 3. Middleware Stack: ? ?
? ? - VerifyCsrfToken ? ?
? ? - Authenticate (`auth`) ? ?
? ? - EnsureActive (custom, valida is_active=true) ? ?
? ? - EnsureAdmin (custom, valida role=admin) ? ?
? ? - RedirectIfActive (en /activate) ? ?
? ? 4. Controller method ? recibe Request, retorna Response ? ?
? ?????????????????????????????????????????????????????????????? ?
? ? ?
? ?????????????????????????????????????????????????????????????? ?
? ? ? ?
? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ? ?
? ? Services ? ?uses??? Eloquent Models ? ?
? ? (lógica de ? ? (Game, Pred, ? ?
? ? negocio) ? ? User, Setting) ? ?
? ?????????????????????????????????????????????????????????????? ?
? ? ? ?
? ? use App\Support\Settings? ?
? ? ? ?
? ?????????????????????????????????????????????????????????????? ?
? ? Blade Views + Components (x-*) ? ?
? ? Layout: layouts/app.blade.php ? ?
? ?????????????????????????????????????????????????????????????? ?
????????????????????????????????????????????????????????????????
? SQL queries (PDO)
?
????????????????????????????????????????????????????????????????
? MYSQL 8 (Hostinger BD) ?
? 8 tablas de negocio + tablas internas Laravel ?
????????????????????????????????????????????????????????????????

????????????????????????????????????????????????????????????????
? CRON HOSTINGER cada minuto: ?
? php artisan schedule:run ?
? ?? predictions:lock (bloquea pronósticos vencidos) ?
? ?? notifications:reminders (recordatorios 15 min antes) ?
????????????????????????????????????????????????????????????????

```

2.2 PATRÓN MVC DE LARAVEL

Laravel implementa MVC con estas convenciones específicas:

- **Modelos** (`app/Models/`): clases que extienden `Illuminate\Database\Eloquent\Model`. Representan registros de la BD y encapsulan reglas de acceso y relaciones (`hasMany`, `belongsTo`, etc.).
- **Vistas** (`resources/views/`): plantillas Blade con extensión `.blade.php`. Reciben datos del controller y renderan HTML.
- **Controllers** (`app/Http/Controllers/`): reciben `Request`, orquestan llamadas a Models/Services, retornan `Response` o `View`.

A los 3 clásicos, este proyecto agrega:

- **Services** (`app/Services/`): lógica de negocio pura, sin dependencias de HTTP. Reutilizable desde controllers y comandos Artisan. Ejemplo: `PredictionScoringService` calcula puntos sin importar quién lo invoque.
- **Components** (`resources/views/components/`): piezas reutilizables de Blade invocadas con `<x-nombre>`.
- **Middleware** (`app/Http/Middleware/`): filtros que se ejecutan antes del controller (auth, validación de estado del usuario, admin-only).
- **Console Commands** (`app/Console/Commands/`): comandos Artisan personalizados invocables por SSH o por el scheduler.

2.3 ESTRUCTURA COMPLETA DE ARCHIVOS DEL PROYECTO

```

soypachonmundial/
?
??? app/
?   ??? Console/
?   ?   ??? Commands/
?   ?   ??? CalculatePredictions.php      ? Comando predictions:calculate {id}
?   ?   ??? LockPredictions.php           ? Comando predictions:lock
?   ?   ??? SendReminderNotifications.php  ? Comando notifications:reminders
?   ?
?   ??? Http/
?   ?   ??? Controllers/
?   ?   ?   ??? Admin/
?   ?   ?   ?   ??? DashboardController.php      ? Métricas del panel admin
?   ?   ?   ?   ??? FixtureController.php        ? CRUD partidos (eliminatória + edición)
?   ?   ?   ?   ??? InvitationCodeController.php  ? Generar / desactivar / exportar códigos
?   ?   ?   ?   ??? ResultsController.php         ? Ingresar resultados + recalcular
?   ?   ?   ?   ??? SettingsController.php        ? Configuración global (premio, video, etc.)
?   ?   ?   ?   ??? UserController.php           ? Gestión de usuarios (toggle activo, buscador)
?   ?   ?   ??? Auth/
?   ?   ?   ?   ??? ActivationController.php      ? Activar código SPM-XXXX
?   ?   ?   ?   ??? LoginController.php          ? Login + logout
?   ?   ?   ?   ??? NewPasswordController.php     ? Reset de contraseña (paso 2)
?   ?   ?   ?   ??? PasswordResetLinkController.php ? Reset de contraseña (paso 1)
?   ?   ?   ?   ??? RegisterController.php        ? Registro con teléfono obligatorio
?   ?   ?   ?   ??? AuditExportController.php     ? Página, CSV y PDF de auditoría
?   ?   ?   ?   ??? Controller.php               ? Base abstracta de Laravel
?   ?   ?   ?   ??? PredictionsController.php     ? Vista, JSON states, save, modal byMatch
?   ?   ?   ?   ??? ProfileController.php         ? Perfil del usuario (editar tel, notifs)
?   ?   ?   ?   ??? RankingController.php         ? Ranking público (index, data JSON, show)
?   ?   ??? Middleware/
?   ?   ?   ??? EnsureActive.php                 ? Redirige a /activate si no is_active
?   ?   ?   ??? EnsureAdmin.php                  ? Bloquea /admin/* a no-admins
?   ?   ?   ??? RedirectIfActive.php              ? Evita /activate si ya activo
?   ?
?   ??? Models/
?   ?   ??? Game.php                            ? tabla "matches" (Match es palabra reservada)
?   ?   ??? MatchNotification.php                ? Idempotencia de recordatorios email
?   ?   ??? Prediction.php                       ? tabla "predictions"
?   ?   ??? Setting.php                          ? tabla "settings" clave-valor
?   ?   ??? User.php                            ? tabla "users", incluye isAdmin()
?   ?
?   ??? Notifications/
?   ?   ??? PredictionReminderNotification.php    ? Mail Notification de Laravel
?   ?
?   ??? Services/
?   ?   ??? PredictionScoringService.php         ? Lógica pura: in 4 ints ? out points
?   ?   ??? PredictionsCalculator.php            ? Orquesta scoring + ranking recalcul
?   ?   ??? RankingService.php                   ? ensureRankingRow + recalculateAll
?   ?
?   ??? Support/
?   ?   ??? Settings.php                         ? Facade-like para BD settings
?   ?
??? bootstrap/
?   ??? app.php                                ? Configuración de la aplicación (middleware aliases)
?   ??? cache/                                  ? caches generados (gitignored)
?
??? config/
?   ?                                           ? Configs de Laravel (database, mail, etc.)
?
??? database/
?   ??? factories/                             ? Solo UserFactory (autogenerado)
?   ??? migrations/
?   ?   ??? 0001_01_01_000000_create_users_table.php      ? users + password_reset_tokens + sessions
?   ?   ??? 0001_01_01_000001_create_cache_table.php      ? cache + cache_locks (Laravel default)
?   ?   ??? 0001_01_01_000002_create_jobs_table.php        ? jobs + job_batches + failed_jobs (Laravel)
?   ?   ??? 2026_05_26_000001_create_matches_table.php     ? matches (Game)
?   ?   ??? 2026_05_26_000002_create_predictions_table.php ? predictions
?   ?   ??? 2026_05_26_000003_create_invitation_codes_table.php ? invitation_codes
?   ?   ??? 2026_05_26_000004_create_rankings_table.php    ? rankings (cache de totales)
?   ?   ??? 2026_05_26_000005_create_settings_table.php    ? settings (clave-valor)
?   ?   ??? 2026_05_26_000006_create_match_notifications_table.php ? match_notifications

```

```

??? docs/
?   ??? informe_tecnico_soypachonmundial.pdf   ? Generado con artisan docs:generate-pdf
?
??? public/
?   ??? build/                                ? Vite output (CSS + JS hasheados) – COMMITTEADO
?   ?   ??? assets/app-*.css
?   ?   ??? assets/app-*.js
?   ?   ??? manifest.json
?   ??? favicon.ico
?   ??? index.php                             ? Front controller de Laravel
?   ??? robots.txt
?
??? resources/
?   ??? css/app.css                           ? Tokens del design system + @tailwind directives
?   ??? js/
?   ?   ??? app.js                             ? Importa Alpine.js
?   ?   ??? bootstrap.js                       ? Axios setup (Laravel default)
?   ??? views/
?   ??? admin/
?   ?   ??? codes/index.blade.php              ? Tabla códigos + generar + exportar (modal)
?   ?   ??? dashboard.blade.php               ? KPIS + premios + top 3 + últimos calculados
?   ?   ??? fixture/edit.blade.php             ? Formulario edición de un partido
?   ?   ??? fixture/index.blade.php            ? Tabla fixture por fase
?   ?   ??? results/index.blade.php            ? Ingresar resultados + recalcular
?   ?   ??? settings/edit.blade.php            ? Form premio, video, mensajes
?   ?   ??? users/index.blade.php              ? Tabla usuarios con buscador
?   ?   ??? _nav.blade.php                     ? Sub-navbar del panel admin
?   ??? audit/
?   ?   ??? index.blade.php                    ? Página con botones Descargar CSV/PDF
?   ?   ??? pdf.blade.php                      ? Template HTML para dompdf
?   ??? auth/
?   ?   ??? activate.blade.php                 ? Form código SPM-XXXX
?   ?   ??? forgot-password.blade.php          ? Form email para reset
?   ?   ??? login.blade.php                    ? Form login email+contraseña
?   ?   ??? register.blade.php                 ? Form registro 5 campos
?   ?   ??? reset-password.blade.php           ? Form nueva contraseña
?   ??? components/
?   ?   ??? badge.blade.php                    ? Variants: default, live, win, upcoming, points
?   ?   ??? btn.blade.php                      ? Variants: primary, accent, ghost, danger, link
?   ?   ??? leaderboard.blade.php              ? Tabla del ranking (Alpine + PHP modes)
?   ?   ??? match-card.blade.php               ? Tarjeta de partido con 3 estados (Alpine)
?   ?   ??? nav.blade.php                      ? Navbar auth + guest
?   ?   ??? stat-card.blade.php                ? KPI card con accent border
?   ??? layouts/
?   ?   ??? app.blade.php                      ? Único layout, head + nav + main + footer
?   ??? predictions/
?   ?   ??? index.blade.php                    ? Vista principal del participante + modal
?   ??? profile/
?   ?   ??? show.blade.php                     ? Datos usuario + editar tel + notifs toggle
?   ??? ranking/
?   ?   ??? index.blade.php                    ? Tabla ranking + premios estimados
?   ?   ??? show.blade.php                     ? Auditoría por usuario + resumen final
?   ??? como-funciona.blade.php               ? Guía pública (6 anchors + calculadora Alpine)
?   ??? welcome.blade.php                      ? Hero simplificado + CTA a /como-funciona
?
??? routes/
?   ??? console.php                           ? Schedule (predictions:lock + notifs cada min)
?   ??? web.php                               ? Todas las rutas HTTP
?
??? storage/
?   ??? app/                                  ? Archivos generados por la app
?   ??? framework/                           ? Cache, sessions, views compiladas
?   ??? logs/laravel.log                      ? Logs de errores y emails (driver log)
?

```

```

??? tests/
?   ??? Feature/
?   ?   ??? Admin/
?   ?   ?   ??? AdminAccessTest.php      ? 4 tests middleware
?   ?   ?   ??? CodesAdminTest.php       ? 5 tests códigos
?   ?   ?   ??? FixtureAdminTest.php      ? 1 test edición eliminatoria
?   ?   ?   ??? ResultsAdminTest.php      ? 2 tests ingreso + recalcular
?   ?   ?   ??? SettingsAdminTest.php     ? 3 tests config
?   ?   ??? AuditExportTest.php           ? 5 tests CSV+PDF
?   ?   ??? AuthFlowTest.php              ? 4 tests flujo registro?activar?login
?   ?   ??? CalculatePredictionsCommandTest.php ? 4 tests comando + desempate
?   ?   ??? LockPredictionsCommandTest.php ? 2 tests bloqueo
?   ?   ??? PredictionsTest.php           ? 10 tests CRUD + byMatch
?   ?   ??? RankingTest.php               ? 11 tests acceso + premios + auditoría
?   ?   ??? RegisterWithPhoneTest.php     ? 3 tests validación teléfono
?   ?   ??? ReminderNotificationsTest.php ? 7 tests emails + idempotencia
?   ?   ??? WelcomeVideoTest.php          ? 5 tests embed video
?   ??? Unit/
?   ?   ??? PredictionScoringServiceTest.php ? 20 casos parametrizados con DataProvider
?
??? vendor/                               ? Composer packages (gitignored)
??? node_modules/                         ? npm packages (gitignored)
?
??? .env                                  ? Variables de entorno LOCAL (gitignored)
??? .env.example                         ? Template para desarrollo
??? .env.production.example              ? Template para producción
??? .gitignore                           ? public/build Sí se commitea (Hostinger sin Node)
??? artisan                              ? CLI de Laravel (php artisan ...)
??? CLAUDE.md                            ? Contexto persistente para sesiones con Claude
??? CHANGELOG.md                         ? Resumen de releases por versión
??? composer.json                        ? Dependencias PHP
??? composer.lock                        ? Lockfile (commiteado)
??? DEPLOY.md                            ? Guía operacional de deploy a Hostinger
??? package.json                         ? Dependencias JS
??? package-lock.json                   ? Lockfile npm (commiteado)
??? phpunit.xml                          ? Config tests (SQLite memoria)
??? README.md                           ? Laravel default
??? tailwind.config.js                   ? Paleta + tokens del design system
??? TECHNICAL_DOCS.md                   ? ESTE DOCUMENTO
??? vite.config.js                       ? Config bundler de assets

```

2.4 FLUJO DE UNA PETICIÓN HTTP

Ejemplo concreto: usuario logueado hace clic en **"Mis Pronósticos"** desde el navbar.

1. Browser envía GET /predictions con cookie de sesión
 - ?
 - ?
2. Apache (Hostinger) recibe en puerto 443 (HTTPS)
 - ? Apache pasa el request a PHP-FPM
 - ?
3. PHP-FPM ejecuta public/index.php (front controller)
 - ? Boot de Laravel: bootstrap/app.php
 - ?
4. Router (routes/web.php) busca match:
 - ? Route::get('/predictions', [PredictionsController::class, 'index'])
 - ? ->middleware(['auth', 'ensure.active'])
 - ?
5. Middleware en orden:
 - ? a. VerifyCsrfToken: GET no requiere CSRF, pasa
 - ? b. Authenticate: cookie válida ? carga el User, sigue. Si no, redirige a /login
 - ? c. EnsureActive: verifica auth()->user()->is_active. Si false, redirige a /activate
 - ?
6. Controller PredictionsController@index(Request \$request)
 - ? - Obtiene los 104 partidos del fixture
 - ? - LEFT JOIN con predictions del usuario
 - ? - Agrupa por fase (grupos, dieciseisavos, ..., final)
 - ? - Calcula \$groups (lista de "A".."L")
 - ? - Retorna view('predictions.index', compact('phases', 'groups'))
 - ?
7. Blade compila resources/views/predictions/index.blade.php
 - ? - Extiende layouts.app
 - ? - Layout incluye <x-nav :user="auth()->user()" />
 - ? - Renderiza el bloque @section('content')
 - ? - Inyecta phases via @js(\$phases) al x-data Alpine
 - ?
8. HTML resultante viaja al browser
 - ? Incluye <link> al CSS + <script type="module"> al JS
 - ?
9. Browser carga public/build/app-{hash}.css (Tailwind compilado)
 - ? Browser carga public/build/app-{hash}.js (Alpine inicializado)
 - ?
10. Alpine.js evalúa x-data="predictionsApp(...)" e inicializa el componente
 - ? Cada match-card se renderiza con sus bindings :class y x-text
 - ? Polling de /predictions/states arranca cada 30s
 - ?
11. Usuario ingresa un score en un input
 - ? @blur dispara save(match)
 - ? Alpine envía POST /predictions/{id} con CSRF token
 - ?
12. Controller PredictionsController@update
 - ? Valida home_score y away_score (0-20)
 - ? Verifica que match.lock_datetime > now()
 - ? Prediction::updateOrCreate(...)
 - ? Retorna JSON {ok: true, prediction: {...}}
 - ?
13. Alpine setea match.savedFlash = true por 1.8s
 - ? Usuario ve "Guardado ?" en el footer del card

2.5 SCHEDULER DE LARAVEL

Laravel tiene un sistema de cron interno (routes/console.php) que registra qué comandos correr y con qué frecuencia. El cron del sistema (Hostinger) lanza UN solo comando cada minuto: `php artisan schedule:run`. Laravel decide internamente qué tareas ejecutar.

Archivo: routes/console.php

```
use Illuminate\Support\Facades\Schedule;

Schedule::command('predictions:lock')
->everyMinute()
->withoutOverlapping() // evita 2 instancias simultáneas
->runInBackground(); // no bloquea el resto del schedule

Schedule::command('notifications:reminders')
->everyMinute()
->withoutOverlapping()
->runInBackground();
```

Comandos programados:

COMANDO	FRECUENCIA	FUNCIÓN
predictions:lock	cada minuto	Marca predictions.is_locked=true y matches.status='live' para partidos cuyo lock_datetime <= NOW()
notifications:reminders	cada minuto	Envía email a usuarios activos sin pronóstico cuya hora de cierre está dentro de los próximos 15 min. Usa tabla match_notifications para idempotencia

Cron de Hostinger:

```
* * * * * cd /home/u123XXXXXX/soypachonmundial && /usr/bin/php artisan schedule:run >> /dev/null 2>&1
```

Para probarlo local sin cron: php artisan schedule:work deja corriendo el scheduler simulando el cron.

3. BASE DE DATOS

3.1 DIAGRAMA ERD

```

????????????????????????????????????????????????????????????????
?                                USERS                                ?
????????????????????????????????????????????????????????????????
? id                            BIGINT PK auto_increment            ?
? name                          VARCHAR(100) NOT NULL              ?
? email                         VARCHAR(150) UNIQUE NOT NULL        ?
? email_verified_at             TIMESTAMP NULL                     ?
? password                      VARCHAR(255) NOT NULL (bcrypt hash) ?
? google_id                     VARCHAR(100) NULL (Fase 3)          ?
? phone_whatsapp                VARCHAR(20) NULL (registro: required) ?
? invitation_code                VARCHAR(20) NULL (código usado al activar) ?
? is_active                     TINYINT(1) DEFAULT 0 (true cuando activa) ?
? role                          ENUM('user','admin') DEFAULT 'user' ?
? notifications_enabled          TINYINT(1) DEFAULT 1              ?
? remember_token                 VARCHAR(100) NULL                  ?
? created_at, updated_at         TIMESTAMP                          ?
????????????????????????????????????????????????????????????????
?
? 1:N
?
?
????????????????????????????????????????????????????????????????
?                                PREDICTIONS                        ?
????????????????????????????????????????????????????????????????
? id                            BIGINT PK auto_increment            ?
? user_id                       BIGINT FK ? users.id ON DELETE CASCADE ?
? match_id                      BIGINT FK ? matches.id ON DELETE CASCADE ?
? home_score                    TINYINT UNSIGNED NOT NULL (0-20)    ?
? away_score                    TINYINT UNSIGNED NOT NULL (0-20)    ?
? points_earned                 TINYINT UNSIGNED NULL              ?
? is_locked                     TINYINT(1) DEFAULT 0              ?
? created_at, updated_at         TIMESTAMP                          ?
? UNIQUE(user_id, match_id)      ?
????????????????????????????????????????????????????????????????
?                                ?                                ?
? N:1                            ? N:1
?                                ?
????????????????????????????????????????????????????????????????
?                                MATCHES (Game)                    ? ? RANKINGS (cache) ?
????????????????????????????????????????????????????????????????
? id                            BIGINT PK auto_increment            ? ? user_id BIGINT PK ?
? phase                         VARCHAR(50)                        ? ? FK ? users.id ?
? (grupos, dieciseisavos, octavos, cuartos, semifinal, ? ? total_points INT DEFAULT 0 ?
? 3er_puesto, final)          ? ? exact_predictions INT DEF 0 ?
? group_name                   VARCHAR(5) NULL (A..L, null en eliminat.) ? ? current_position INT NULL ?
? match_number                 INT UNIQUE NOT NULL (1..104)         ? ? previous_position INT NULL ?
? home_team                    VARCHAR(60)                          ? ? last_calculated_at TIMESTAMP ?
? away_team                    VARCHAR(60)                          ? ? created_at, updated_at ?
? home_flag                     VARCHAR(10) NULL (emoji bandera) ? ?????????????????????????????????
? away_flag                     VARCHAR(10) NULL                  ?
? match_datetime               DATETIME NOT NULL                  ?
? venue                        VARCHAR(100) NULL (Estadio, Ciudad) ?
? status                       ENUM('upcoming','live','finished') ?
? home_score_official          TINYINT UNSIGNED NULL              ?
? away_score_official          TINYINT UNSIGNED NULL              ?
? lock_datetime                DATETIME NOT NULL (= match - 5 min) ?
? api_match_id                 VARCHAR(50) NULL (para API-Football F3) ?
? created_at, updated_at         TIMESTAMP                          ?
????????????????????????????????????????????????????????????????
?
? 1:N
?
????????????????????????????????????????????????????????????????
?                                MATCH_NOTIFICATIONS                ?

```

```

????????????????????????????????????????????????????????????
? id                BIGINT PK auto_increment                ?
? user_id           BIGINT FK ? users.id ON DELETE CASCADE  ?
? match_id          BIGINT FK ? matches.id ON DELETE CASCADE ?
? type              VARCHAR(30) ('reminder')                ?
? sent_at           TIMESTAMP NOT NULL                       ?
? created_at, updated_at  TIMESTAMP                         ?
? UNIQUE(user_id, match_id, type) ? garantiza idempotencia ?
????????????????????????????????????????????????????????????

????????????????????????????????????????????????????????????
?                INVITATION_CODES                          ?
????????????????????????????????????????????????????????????
? id                BIGINT PK auto_increment                ?
? code              VARCHAR(20) UNIQUE NOT NULL (SPM-XXXX)  ?
? is_used           TINYINT(1) DEFAULT 0                    ?
? used_by_user_id   BIGINT FK ? users.id ON DELETE SET NULL ?
? used_at           TIMESTAMP NULL                           ?
? is_active         TINYINT(1) DEFAULT 1                     ?
? created_at, updated_at  TIMESTAMP                         ?
????????????????????????????????????????????????????????????

????????????????????????????????????????????????????????????
?                SETTINGS                                    ?
????????????????????????????????????????????????????????????
? id                BIGINT PK auto_increment                ?
? key               VARCHAR(60) UNIQUE NOT NULL              ?
? value            TEXT NULL                                 ?
? created_at, updated_at  TIMESTAMP                         ?
????????????????????????????????????????????????????????????
Keys conocidas:
- prize_pool        (entero, COP)
- tournament_name   (string)
- welcome_message   (string)
- video_url         (URL YouTube)

???? Tablas internas de Laravel (no documentadas en detalle) ???
? password_reset_tokens · sessions · cache · cache_locks    ?
? jobs · job_batches · failed_jobs · migrations             ?
????????????????????????????????????????????????????????

```

3.2 DETALLE DE CADA TABLA DE NEGOCIO

users

Tabla principal de identidades.

CAMPO	TIPO	NOTAS
id	BIGINT PK	autoincrement
name	VARCHAR(100)	"Nombre Apellido" concatenado en registro
email	VARCHAR(150) UNIQUE	login + recipient de emails
email_verified_at	TIMESTAMP	en MVP se setea <code>now()</code> al registrar (sin verificación real)
password	VARCHAR(255)	bcrypt hash (rounds=12 según config)
google_id	VARCHAR(100) NULL	reservado para OAuth Fase 3
phone_whatsapp	VARCHAR(20)	obligatorio, regex <code>[0-9]{7,15}</code>
invitation_code	VARCHAR(20) NULL	código SPM-XXXX que usó al activar
is_active	BOOL DEFAULT 0	true cuando activó el código. Solo activos ven <code>/predictions</code> y <code>/ranking</code>
role	ENUM('user','admin')	admin tiene acceso a <code>/admin/*</code>
notifications_enabled	BOOL DEFAULT 1	controla recepción de emails
remember_token	VARCHAR(100)	para checkbox "Recordarme"

`matches` (modelo `App\Models\Game`)

Fixture completo: 104 partidos. **Nota:** el modelo se llama `Game` porque `Match` es palabra reservada en PHP 8.

CAMPO	TIPO	NOTAS
id	BIGINT PK	
phase	VARCHAR(50)	uno de: <code>grupos</code> , <code>dieciseisavos</code> , <code>octavos</code> , <code>cuartos</code> , <code>semifinal</code> , <code>3er_puesto</code> , <code>final</code>
group_name	VARCHAR(5) NULL	A..L, NULL en eliminatoria
match_number	INT UNIQUE	1..104, número oficial FIFA
home_team / away_team	VARCHAR(60)	nombres de equipos
home_flag / away_flag	VARCHAR(10) NULL	emoji <code>🇺🇸</code>
match_datetime	DATETIME	fecha/hora de inicio (interpretada en <code>America/Bogota</code>)
venue	VARCHAR(100)	"Estadio, Ciudad" o "Por definir"
status	ENUM('upcoming','live','finished')	transiciones manejadas por comandos
home_score_official / away_score_official	TINYINT NULL	NULL hasta que el admin ingrese el resultado
lock_datetime	DATETIME	calculado: <code>match_datetime - 5 min</code>
api_match_id	VARCHAR(50) NULL	reservado para API-Football Fase 3

predictions

Una fila por pronóstico de cada usuario para cada partido.

CAMPO	TIPO	NOTAS
id	BIGINT PK	
user_id	BIGINT FK	cascade on delete
match_id	BIGINT FK	cascade on delete
home_score / away_score	TINYINT 0-20	NOT NULL — el formulario garantiza ambos
points_earned	TINYINT NULL	NULL hasta que se calcula
is_locked	BOOL	cacheado por predictions:lock
UNIQUE	(user_id, match_id)	un solo pronóstico por user/partido

invitation_codes

Códigos generados por el admin.

CAMPO	TIPO	NOTAS
id	BIGINT PK	
code	VARCHAR(20) UNIQUE	formato SPM-XXXX (alfabeto sin 0/O/I/1/L)
is_used	BOOL	true cuando alguien lo activó
used_by_user_id	BIGINT FK NULL	quién lo usó (ON DELETE SET NULL)
used_at	TIMESTAMP NULL	cuándo
is_active	BOOL	admin puede desactivar un código no usado

rankings

Tabla cache/snapshot del ranking. Una fila por usuario.

CAMPO	TIPO	NOTAS
user_id	BIGINT PK	también es FK
total_points	INT DEFAULT 0	suma de predictions.points_earned
exact_predictions	INT DEFAULT 0	cantidad con points_earned = 5
current_position	INT NULL	calculada por RankingService::recalculateAll
previous_position	INT NULL	snapshot anterior, para detectar subida/bajada
last_calculated_at	TIMESTAMP NULL	timestamp del último recálculo

settings

Clave-valor genérico para configuración global.

CAMPO	TIPO	NOTAS
id	BIGINT PK	
key	VARCHAR(60) UNIQUE	
value	TEXT NULL	

Claves conocidas:

- prize_pool → entero COP (e.g. 500000)
- tournament_name → string público
- welcome_message → string público
- video_url → URL completa de YouTube (watch,youtu.be o embed)

match_notifications

Idempotencia de envíos de email.

CAMPO	TIPO	NOTAS
id	BIGINT PK	
user_id	BIGINT FK	
match_id	BIGINT FK	
type	VARCHAR(30)	actualmente solo 'reminder'
sent_at	TIMESTAMP	
UNIQUE	(user_id, match_id, type)	el comando inserta primero, envía después → race-safe

3.3 ÍNDICES Y CLAVES FORÁNEAS

TABLA	ÍNDICES
users	UNIQUE(email)
matches	UNIQUE(match_number), INDEX(phase), INDEX(status), INDEX(match_datetime)
predictions	UNIQUE(user_id, match_id), INDEX(match_id), FK user_id, FK match_id
invitation_codes	UNIQUE(code), INDEX(is_used), INDEX(is_active), FK used_by_user_id
rankings	PK(user_id), INDEX(total_points), INDEX(current_position), FK user_id
settings	UNIQUE(key)
match_notifications	UNIQUE(user_id, match_id, type), FK user_id, FK match_id

3.4 MIGRACIONES EN ORDEN CRONOLÓGICO

```
1. 0001_01_01_000000_create_users_table.php
2. 0001_01_01_000001_create_cache_table.php
3. 0001_01_01_000002_create_jobs_table.php
4. 2026_05_26_000001_create_matches_table.php
5. 2026_05_26_000002_create_predictions_table.php
6. 2026_05_26_000003_create_invitation_codes_table.php
7. 2026_05_26_000004_create_rankings_table.php
8. 2026_05_26_000005_create_settings_table.php
9. 2026_05_26_000006_create_match_notifications_table.php
```

Estado verificable:

```
php artisan migrate:status
```

3.5 BACKUP EN HOSTINGER PREMIUM

Backup manual diario recomendado

Desde SSH:

```
mysqldump -u u123XXXXXX_pachon_user -p u123XXXXXX_pachon \
> ~/backups/spm-$(date +%F-%H%M).sql
```

Compresión + retención:

```
mkdir -p ~/backups
mysqldump -u ${DB_USER} -p${DB_PASS} ${DB_NAME} | gzip > ~/backups/spm-$(date +%F).sql.gz
# Mantener solo últimos 14 días:
find ~/backups -name "spm-*.sql.gz" -mtime +14 -delete
```

Programar como cron en hPanel:

```
0 3 * * * mysqldump -u ${USER} -p${PASS} ${DB} | gzip > ~/backups/spm-$(date +%F).sql.gz; find ~/backups -name "spm-*.sql.gz" -mtime +14 -delete
```

Backup automático de Hostinger

Hostinger Premium incluye respaldo automático semanal. Verificable en hPanel → **Archivos** → **Copias de seguridad**. Suele guardar los últimos 7 días pero verificar política vigente.

Restauración

```
gunzip -c ~/backups/spm-2026-06-15.sql.gz | mysql -u ${USER} -p${PASS} ${DB}
```

3.6 QUERIES ÚTILES PARA DIAGNÓSTICO

1. Ranking actual ordenado:


```
SELECT r.current_position AS pos, u.name, r.total_points AS pts, r.exact_predictions AS exactos
FROM rankings r
JOIN users u ON u.id = r.user_id
WHERE u.is_active = 1 AND u.role = 'user'
ORDER BY r.current_position;
```

2. Pronósticos de un usuario específico:

```
SELECT m.match_number, m.home_team, m.away_team,
       p.home_score, p.away_score,
       m.home_score_official, m.away_score_official,
       p.points_earned, m.status
FROM predictions p
JOIN matches m ON m.id = p.match_id
JOIN users u ON u.id = p.user_id
WHERE u.email = 'carlos@demo.com'
ORDER BY m.match_number;
```

3. Partidos sin resultado ingresado (candidatos a ser calculados):

```
SELECT m.id, m.match_number, m.home_team, m.away_team,
       m.match_datetime, m.status
FROM matches m
WHERE m.status IN ('upcoming', 'live')
      AND m.match_datetime <= NOW()
ORDER BY m.match_datetime;
```

4. Códigos de invitación disponibles para enviar:

```
SELECT code
FROM invitation_codes
WHERE is_used = 0 AND is_active = 1
ORDER BY code;
```

5. Recalcular manualmente puntos de un partido (combina SQL + Artisan):

```
# Por SSH al servidor:
php artisan predictions:calculate {match_id}
```

o vía panel admin en `/admin/resultados` clickeando "Recalcular".

Otras queries útiles:

```
-- Distribución de puntos por partido finalizado
SELECT m.match_number,
       SUM(p.points_earned = 5) AS p5,
       SUM(p.points_earned = 3) AS p3,
       SUM(p.points_earned = 2) AS p2,
       SUM(p.points_earned = 1) AS p1,
       SUM(p.points_earned = 0) AS p0
FROM matches m
JOIN predictions p ON p.match_id = m.id
WHERE m.status = 'finished'
GROUP BY m.id, m.match_number
ORDER BY m.match_number;

-- Usuarios pendientes de activar (registrados sin código)
SELECT id, name, email, phone_whatsapp, created_at
FROM users
WHERE is_active = 0 AND role = 'user'
ORDER BY created_at DESC;

-- Verificar settings activos
SELECT `key`, `value` FROM settings ORDER BY `key`;
```

4. MÓDULOS FUNCIONALES

4.1 MÓDULO DE AUTENTICACIÓN

Propósito: Permitir registro de cuentas, login con email/contraseña, activación mediante código de invitación, recuperación de contraseña.

Archivos involucrados:

- app/Http/Controllers/Auth/RegisterController.php
- app/Http/Controllers/Auth/LoginController.php
- app/Http/Controllers/Auth/ActivationController.php
- app/Http/Controllers/Auth/PasswordResetLinkController.php
- app/Http/Controllers/Auth/NewPasswordController.php
- app/Http/Middleware/EnsureActive.php
- app/Http/Middleware/RedirectIfActive.php
- app/Models/User.php
- resources/views/auth/*.blade.php (5 vistas)
- routes/web.php (grupos guest y auth)

Flujo de registro:

1. Usuario abre /register
2. Completa: nombre, apellido, email, teléfono (7-15 dígitos), password
3. POST /register ? RegisterController@store
 - Valida: nombre min 2/max 50, email único, telefono regex
 - User::create() con is_active=false, email_verified_at=now()
 - Auth::login(\$user)
 - Redirige a /activate
4. Usuario ingresa código SPM-XXXX en /activate
5. POST /activate ? ActivationController@store
 - DB::transaction:
 - a. Busca invitation_code con is_active=true AND is_used=false
 - b. Si existe: marca is_used=true, used_by_user_id, used_at
 - c. Actualiza user.is_active=true, user.invitation_code
 - Crea fila en rankings con 0/0 via RankingService
 - Redirige a /predictions

Flujo de login:

1. Usuario abre /login
2. Completa email + password
3. POST /login ? LoginController@store
 - Auth::attempt() con remember opcional
 - Si is_active=false ? /activate
 - Si is_active=true ? /predictions (intended)

Recuperación de contraseña usa el sistema built-in de Laravel:

- /forgot-password → envía email con link a /reset-password/{token}?email=...
- En desarrollo el email se escribe al log: storage/logs/laravel.log
- En producción se envía vía SMTP a la dirección del usuario

Errores comunes:

- **"Las credenciales son incorrectas"** → password incorrecto o email no registrado. Verificar en BD: `SELECT email FROM users WHERE email='...'`
- **"Código incorrecto o ya utilizado"** → el código no existe, ya fue usado o fue desactivado. Verificar en BD: `SELECT * FROM invitation_codes WHERE code='SPM-XXXX'`
- **Usuario queda en bucle en /activate** → su `is_active` no se actualizó correctamente. Solución: `UPDATE users SET is_active=1 WHERE email='...'` y crear fila en rankings
- **No le llega email de reset** → ver sección 9 sobre emails

4.2 MÓDULO DE FIXTURE Y PARTIDOS

Propósito: Mantener el catálogo de 104 partidos del Mundial, permitir edición y consulta.

Archivos involucrados:

- database/seeders/MatchSeeder.php — 72 partidos fase grupos con datos reales
- database/seeders/EliminatorySeeder.php — 32 partidos eliminatória como placeholder
- app/Models/Game.php — modelo Eloquent
- app/Http/Controllers/Admin/FixtureController.php — CRUD desde admin
- resources/views/admin/fixture/index.blade.php — tabla por fase
- resources/views/admin/fixture/edit.blade.php — formulario edición

Inicialización: Los 104 partidos se cargan vía `php artisan db:seed` (corre `MatchSeeder` y `EliminatorySeeder`).

Edición desde admin:

```
/admin/fixture ? tabla con todos los partidos agrupados por fase
? "Editar" en cada fila ? /admin/fixture/{id}/editar
? Form con: home_team, away_team, banderas, fecha, hora, venue
? PATCH /admin/fixture/{id} ? recalcula lock_datetime
```

Cuándo editar:

- Partidos de **fase de grupos**: NO suele cambiar (FIFA lo publica oficial)
- Partidos de **eliminatória**: tras cada fase, reemplazar "Clasificado A1" por el equipo real
- Si FIFA cambia fecha/hora/venue de un partido: actualizar

Errores comunes:

- Editar un partido ya **finished** rompe los puntos calculados → re-correr `predictions:calculate` después
- Cambiar `match_datetime` no recalcula puntos de pronósticos viejos → verificar `lock_datetime` también

4.3 MÓDULO DE PRONÓSTICOS Y BLOQUEO AUTOMÁTICO

Propósito: Permitir a cada participante ingresar marcadores, garantizar el cierre automático 5 min antes del partido.

Archivos involucrados:

- `app/Http/Controllers/PredictionsController.php` — index, update, states, byMatch
- `app/Models/Prediction.php`
- `app/Console/Commands/LockPredictions.php` — comando `predictions:lock`
- `resources/views/predictions/index.blade.php` — vista con Alpine
- `resources/views/components/match-card.blade.php` — componente reutilizable

Flujo de guardado:

```
1. Usuario activa input de score en /predictions
2. Al desenfocar (@blur), Alpine envía POST /predictions/{id}
   con {home_score: N, away_score: M}
3. Controller valida:
   - Ambos campos enteros 0..20
   - Match.lock_datetime > now() (no bloqueado)
4. Prediction::updateOrCreate(user_id+match_id, scores)
5. Retorna JSON {ok: true, prediction: {...}}
6. Alpine muestra "Guardado ?" en footer del card 1.8s
```

Polling automático: Cada 30s Alpine consulta `/predictions/states` y actualiza estados de cada partido (bloqueado/finalizado/scores oficiales). Útil cuando el `predictions:lock` corre en background.

Modal "Ver Pronósticos": Botón visible solo en partidos `is_locked` o `finished`. Llama a `/partidos/{id}/pronosticos` (endpoint JSON). Muestra todos los usuarios activos con sus pronósticos.

Comando de bloqueo:

```
# Lo corre el scheduler cada minuto:
php artisan predictions:lock

# Output: "Partidos pasados a 'live': N / Pronósticos bloqueados: M"
```

Errores comunes:

- Usuario reporta "no se guarda mi pronóstico" → verificar CSRF token. Si la sesión expiró, el POST devuelve 419. Recargar la página.
- "El campo no se bloquea aunque ya pasó la hora" → cron no está corriendo. Ver sección 9.
- Modal Ver Pronósticos no abre → es Alpine. Si hay error JS, abrir DevTools console. Históricamente fue bug con `$root` (resuelto en commit a98b59c).

4.4 MOTOR DE CÁLCULO DE PUNTOS (PREDICTIONSCORINGSERVICE)

Propósito: Aplicar la tabla de puntuación 5/3/2/1/0 a cada pronóstico.

Archivos involucrados:

- `app/Services/PredictionScoringService.php` — lógica pura (in: 4 ints, out: 1 int)
- `app/Services/PredictionsCalculator.php` — orquesta scoring + ranking
- `app/Console/Commands/CalculatePredictions.php` — comando CLI
- `app/Http/Controllers/Admin/ResultsController.php` — invoca desde admin
- `tests/Unit/PredictionScoringServiceTest.php` — 20 casos parametrizados

Tabla de puntos (regla estricta same-side):

PTS	CONDICIÓN
5	<code>home_pred == home_off</code> Y <code>away_pred == away_off</code>
3	Ganador correcto Y exactamente un marcador exacto en su lado
2	Ganador correcto, ningún marcador exacto en su lado
1	Ganador incorrecto Y al menos un marcador exacto en su lado (XOR)
0	Cualquier otra cosa, incluyendo "espejo cruzado" (pred 1-2 vs result 2-1 → 0)

Código del scoring (referencia, no editar a la ligera):

```

public function calculate(int $predHome, int $predAway, int $offHome, int $offAway): int
{
    $exactHome = $predHome === $offHome;
    $exactAway = $predAway === $offAway;
    if ($exactHome && $exactAway) return 5;

    $predWinner = $predHome <=> $predAway;
    $offWinner = $offHome <=> $offAway;
    $winnerCorrect = $predWinner === $offWinner;

    if ($winnerCorrect && ($exactHome || $exactAway)) return 3;
    if ($winnerCorrect) return 2;
    if ($exactHome || $exactAway) return 1;
    return 0;
}

```

Cuándo se ejecuta:

1. Admin ingresa resultado oficial en `/admin/resultados` → `ResultsController@store` invoca `PredictionsCalculator::calculate($game)`
2. Manualmente por CLI: `php artisan predictions:calculate {id}`

Lo que hace `PredictionsCalculator::calculate`:

1. Verifica que `match.home_score_official` y `away_score_official` NO sean null
2. `DB::transaction`:
 - Itera todas las predictions de ese match
 - Calcula points con `PredictionScoringService`
 - `UPDATE predictions SET points_earned = N`
3. Si `match.status != 'finished'` ? `UPDATE matches SET status='finished'`
4. Invoca `RankingService::recalculateAll()`
5. Retorna `{match_number, predictions_count, distribution{5,3,2,1,0}, ranking_count, top[5]}`

Errores comunes:

- "El partido no tiene resultado oficial cargado" → ingresar primero `home_score_official` y `away_score_official` en BD o vía admin
- Después de cambiar la regla de scoring, los puntos viejos quedan calculados con la regla anterior → recalculan todos los partidos finalizados manualmente

4.5 MÓDULO DE RANKING Y DESEMPATE

Propósito: Mantener una tabla snapshot del ranking con desempate por exactos.

Archivos involucrados:

- `app/Services/RankingService.php` — `ensureRankingRow` + `recalculateAll`
- `app/Http/Controllers/RankingController.php` — `index`, `data`, `show`
- `app/Models/User.php` (relación rankings)
- `resources/views/ranking/index.blade.php` — tabla pública
- `resources/views/ranking/show.blade.php` — auditoría por usuario
- `resources/views/components/leaderboard.blade.php` — componente tabla

Lógica de recálculo (`RankingService::recalculateAll`):

- 1. Snapshot de previous_position desde rankings
- 2. Garantiza fila en rankings para cada user activo (ensureRankingRow)
- 3. Query SQL:

SELECT user_id, name,
SUM(p.points_earned) AS total_points,
SUM(CASE WHEN p.points_earned=5 THEN 1 ELSE 0 END) AS exactos
FROM users LEFT JOIN predictions ON ...
WHERE users.is_active=1
GROUP BY user_id
- 4. Ordenamiento: total_points DESC, exactos DESC, user_id ASC
- 5. Asignación de current_position con manejo de empates (posición compartida)
- 6. UPDATE rankings con nuevos valores + last_calculated_at=now()

Visualización pública (/ranking , requiere usuario activo):

- Tarjetas de acumulado y premios 60/25/15 arriba
- Tabla con [] en las 3 primeras posiciones
- Botón "Ver pronósticos" por fila → navega a /ranking/u/{id}

Auditoría por usuario (/ranking/u/{id}):

- Solo muestra partidos status=finished con points_earned NOT NULL
- Header con posición, puntos, exactos
- Resumen al final: Total puntos, Exactos, Partidos jugados N/M, Aprovechamiento %
- Botón "Volver al Ranking" arriba y abajo (mobile-friendly)

Errores comunes:

- Ranking desactualizado después de un cambio manual en BD → forzar php artisan tinker y app(\App\Services\RankingService::class)->recalculateAll()
- Usuario aparece con 0 puntos pero tiene predictions con points_earned → su is_active está en 0. Solución: activarlo

4.6 PANEL DE ADMINISTRACIÓN

Propósito: Permitir al administrador gestionar el torneo en vivo.

Acceso: /admin — protegido por middleware auth + ensure.active + admin . Solo usuarios con role='admin' entran.

6 vistas del panel:

RUTA	CONTROLLER	PROPÓSITO
/admin	Admin/DashboardController	KPIs, top 3, últimos calculados, premios
/admin/codigos	Admin/InvitationCodeController	Generar SPM-XXXX, listar, desactivar, exportar
/admin/usuarios	Admin/UserController	Listar con buscador, toggle activo
/admin/fixture	Admin/FixtureController	Editar equipos / fecha / venue de cualquier partido
/admin/resultados	Admin/ResultsController	CRÍTICA — ingresar resultado oficial + recalcular
/admin/configuracion	Admin/SettingsController	Acumulado, nombre torneo, mensaje welcome, URL video

Detalle de cada vista en sección 7.

Errores comunes:

- Usuario normal accede a `/admin` y ve mensaje "No tenés permisos" → comportamiento esperado, no es bug
- Admin no ve los KPIs correctos → cache de queries puede estar stale. Refrescar.

4.7 MÓDULO DE NOTIFICACIONES POR EMAIL

Propósito: Avisar a usuarios sin pronóstico 15 min antes del cierre.

Archivos involucrados:

- `app/Notifications/PredictionReminderNotification.php` — clase Laravel Notification
- `app/Console/Commands/SendReminderNotifications.php` — comando `notifications:reminders`
- `app/Models/MatchNotification.php` — idempotencia
- `routes/console.php` — scheduler

Flujo del comando (corre cada minuto):

```
1. window = [now(), now()+15min]
2. Busca matches con status=upcoming AND lock_datetime BETWEEN window
3. Para cada match:
  a. Busca users con is_active=1 AND notifications_enabled=1
    AND no tiene prediction para este match
    AND no tiene match_notification(type='reminder') para este match
  b. Para cada user candidato:
    - DB::transaction:
      * INSERT INTO match_notifications (user, match, type, sent_at)
        ? si la UNIQUE constraint falla (race), salta
    - Si insert OK: $user->notify(new PredictionReminderNotification($match))
4. Output: "Recordatorios enviados: N"
```

Cómo se envía el email:

- Driver `log` (desarrollo): se escribe a `storage/logs/laravel.log`
- Driver `smtp` (producción): se envía vía SMTP de Hostinger Mail

Contenido del email: ver `PredictionReminderNotification::toMail()`. Incluye saludo personalizado, nombre del partido con banderas, fecha en español, botón "Ir a Mis Pronósticos", mensaje sobre 0 puntos.

Errores comunes:

- Emails no se envían en producción → ver sección 9 (verificar SMTP)
- Mismo usuario recibe el email 2 veces → no debería pasar por la UNIQUE constraint. Si pasa, revisar logs.
- Usuario quiere desactivar emails → en `/perfil`, toggle "Recibir notificaciones por email"

4.8 MÓDULO DE AUDITORÍA Y EXPORTACIÓN

Propósito: Exponer transparencia total — cualquiera puede descargar el detalle de pronósticos calculados.

Archivos involucrados:

- `app/Http/Controllers/AuditExportController.php`
- `resources/views/audit/index.blade.php` — página con 2 botones
- `resources/views/audit/pdf.blade.php` — template PDF (dompdf)

Rutas:

- `/auditoria/exportar` — página con botones (auth + active)
- `/auditoria/exportar/csv` — descarga CSV (auth + active)
- `/auditoria/exportar/pdf` — descarga PDF (auth + active)
- `/admin/auditoria/exportar` — misma página desde admin con sub-nav

Filtros aplicados:

- `matches.status='finished'`
- `predictions.points_earned IS NOT NULL`
- `users.is_active=1`

Columnas: Partido | Fase | Fecha | Resultado oficial | Usuario | Pronóstico | Puntos.

Ordenamiento: `match_datetime ASC, points_earned DESC, user_name ASC`.

Formato CSV:

- UTF-8 con BOM al inicio (para Excel)
- `Content-Type: text/csv; charset=UTF-8`
- Nombre: `SoyPachonMundial_Auditoria_YYYY-MM-DD.csv`

Formato PDF:

- Letter landscape
- DejaVu Sans (dompdf default) + Courier New para marcadores
- Badges de puntos coloreados según puntaje
- Filas alternadas con bg `#faf7ef`
- Header con brand, footer con total y URL
- Nombre: `SoyPachonMundial_Auditoria_YYYY-MM-DD.pdf`

Errores comunes:

- Descarga vacía → no hay partidos finished con puntos calculados aún
- PDF genera error de memoria con muchas filas → ajustar `php.ini` `memory_limit` (Hostinger default suele ser 512M, suficiente para ~5000 filas)

4.9 SECCIÓN PÚBLICA "¿CÓMO FUNCIONA?"

Propósito: Auto-servicio para nuevos usuarios. Reduce la carga de soporte explicando todo lo que necesitan saber.

Archivos:

- `routes/web.php` → `Route::view('/como-funciona', 'como-funciona')`
- `resources/views/como-funciona.blade.php`

Secciones (anchors):

1. `#video` — embed YouTube desde `Settings::videoEmbedUrl()`
2. `#puntos` — tabla 5/3/2/1/0 con colorimetría unificada
3. `#premio` — distribución 60/25/15 + calculadora Alpine
4. `#pronosticos` — 5 pasos del flujo
5. `#ranking` — explicación + tabla ejemplo
6. `#inscripcion` — 4 pasos (crear cuenta → pagar → enviar comprobante → activar) + botón WhatsApp

Calculadora Alpine:

```
function prizeCalculator(cupo) {  
    return {  
        participants: 20,  
        cupo: cupo,  
        formatMoney(n) { return new Intl.NumberFormat('es-CO',{style:'currency',currency:'COP'}).format(n); },  
    };  
}
```

Slider + input numérico sincronizados. Calcula en vivo el premio por puesto.

Errores comunes:

- Anchor links no scrollean suavemente → revisar que `html { scroll-behavior: smooth }` esté en `app.css`
- Botón WhatsApp no abre la app → URL del esquema: `https://wa.me/573013966515` (sin +)

5. APIS Y SERVICIOS EXTERNOS

5.1 GMAIL SMTP / HOSTINGER SMTP (MVP — ACTIVO)

Propósito: Envío de emails transaccionales (recuperación de contraseña, recordatorios de pronóstico).

Credenciales necesarias:

- Email completo (e.g. `notificaciones@soypachonmundial.online`)
- Password del email
- Host SMTP del proveedor

Configuración en `.env`:

```
MAIL_MAILER=smtp
MAIL_HOST=smtp.hostinger.com
MAIL_PORT=465
MAIL_USERNAME=notificaciones@soypachonmundial.online
MAIL_PASSWORD=[CONFIGURAR EN .env]
MAIL_ENCRYPTION=ssl
MAIL_FROM_ADDRESS=notificaciones@soypachonmundial.online
MAIL_FROM_NAME="${APP_NAME}"
```

Para usar Gmail en lugar de Hostinger Mail:

```
MAIL_HOST=smtp.gmail.com
MAIL_PORT=587
MAIL_ENCRYPTION=tls
MAIL_USERNAME=[gmail]
MAIL_PASSWORD=[App Password generada en https://myaccount.google.com/apppasswords]
```

Cómo probar que funciona:

```
php artisan tinker
>>> Mail::raw('Prueba SMTP', function($m){ $m->to('vos@email.com')->subject('Test'); });
```

Revisar inbox. Si no llega:

- Verificar credenciales con webmail directamente
- Ver `storage/logs/laravel.log` por error SMTP
- Hostinger a veces tiene puerto 587 / TLS en vez de 465 / SSL

Qué hacer si falla:

- Cambiar driver temporalmente a `log` para no romper la app (`MAIL_MAILER=log`)
- Verificar que la cuenta de correo esté activa en hPanel
- Si es Gmail: regenerar el App Password (no usar la pass de la cuenta principal)

5.2 API-FOOTBALL (FASE 3 — PENDIENTE)

Propósito: Sincronizar resultados oficiales del Mundial automáticamente sin intervención del admin.

Servicio sugerido: <https://www.api-football.com/> vía RapidAPI.

Credenciales necesarias:

- API key (de RapidAPI)
- Plan: el free tier permite 100 requests/día (suficiente para checar resultados 2x al día)

Configuración en `.env` :

```
API_FOOTBALL_KEY=[CONFIGURAR EN .env]
API_FOOTBALL_HOST=v3.football.api-sports.io
API_FOOTBALL_LEAGUE_ID=1 # World Cup 2026 (verificar al activarlo)
```

Implementación pendiente:

1. Crear `app/Services/ApiFootballService.php` con métodos:

- `fetchMatchResults(int $apiMatchId): ?array`
- `syncAllMatches(): int` (recorre matches no finished con `api_match_id` setado)

2. Crear comando `app/Console/Commands/SyncResults.php`:

- Llama a `syncAllMatches()`
- Por cada match con resultado nuevo: actualiza `home_score_official`, `away_score_official`, `status='finished'`
- Invoca `PredictionsCalculator::calculate()`

3. Agregar a scheduler cada 10 min:

```
Schedule::command('results:sync')->everyTenMinutes();
```

4. Mapeo `match_number ? api_match_id`: poblar la columna `matches.api_match_id` con los IDs de API-Football vía seeder de migración

Cómo probar:

```
curl -H "x-rapidapi-key: $API_FOOTBALL_KEY" \  
-H "x-rapidapi-host: $API_FOOTBALL_HOST" \  
"https://v3.football.api-sports.io/fixtures?id=12345"
```

Qué hacer si falla:

- Verificar quota: el dashboard de RapidAPI muestra cuántos requests llevás
- Fallback: el admin sigue pudiendo ingresar resultados manualmente desde `/admin/resultados`. La integración API es solo para reducir su carga.

5.3 GOOGLE OAUTH (FASE 3 — PENDIENTE)

Propósito: Permitir login con cuenta Google además de email/contraseña.

Credenciales necesarias:

- Client ID y Client Secret de Google Cloud Console (<https://console.cloud.google.com/>)
- OAuth consent screen aprobado por Google

Configuración en `.env`:

```
GOOGLE_CLIENT_ID=[CONFIGURAR EN .env]  
GOOGLE_CLIENT_SECRET=[CONFIGURAR EN .env]  
GOOGLE_REDIRECT_URI=https://soypachonmundial.online/auth/google/callback
```

Implementación pendiente:

1. Instalar: `composer require laravel/socialite`
2. Agregar a `config/services.php`:

```
'google' => [
    'client_id' => env('GOOGLE_CLIENT_ID'),
    'client_secret' => env('GOOGLE_CLIENT_SECRET'),
    'redirect' => env('GOOGLE_REDIRECT_URI'),
],
```

3. Crear rutas:

```
Route::get('/auth/google', [GoogleController::class, 'redirect']);
Route::get('/auth/google/callback', [GoogleController::class, 'callback']);
```

4. Crear `app/Http/Controllers/Auth/GoogleController.php`:

- `redirect()` → `Socialite::driver('google')->redirect()`
- `callback()` → busca user por email o `google_id`, si no existe crea uno con `is_active=false` (igual flujo: necesita código)

5. Botón "Iniciar sesión con Google" en `auth/login.blade.php`

Cómo probar: click en el botón Google desde `/login`. Verificar que llegue al callback con email correcto.

Qué hacer si falla:

- Verificar que el redirect URI en Google Console coincida EXACTO con el `.env`
- Si Google muestra "This app isn't verified": en pruebas internas agregar el email como test user en OAuth consent screen

5.4 CALLMEBOT WHATSAPP (FASE 3 — PENDIENTE)

Propósito: Reemplazar/complementar emails con notificaciones WhatsApp.

Credenciales necesarias:

- Por usuario: el usuario debe activar el bot enviando un mensaje específico a CallMeBot desde su WhatsApp para obtener su `api_key`. Ver <https://www.callmebot.com/blog/free-api-whatsapp-messages/>

Almacenamiento: agregar columna `users.callmebot_api_key` (string nullable).

Implementación pendiente:

1. Migración: `ALTER TABLE users ADD callmebot_api_key VARCHAR(100) NULL;`
2. En `/perfil` agregar input para pegar el `api_key` + instrucciones
3. Crear `app/Services/WhatsAppService.php`:

```
public function send(User $user, string $message): bool
{
    if (!$user->callmebot_api_key || !$user->phone_whatsapp) return false;
    $url = 'https://api.callmebot.com/whatsapp.php?'
        . http_build_query([
            'phone' => '57' . $user->phone_whatsapp,
            'text' => $message,
            'apikey' => $user->callmebot_api_key,
        ]);
    $response = Http::get($url);
    return $response->successful();
}
```

4. Modificar `SendReminderNotifications` para usar este servicio en lugar (o además) del email

Cómo probar: ingresar manualmente el api_key de un usuario test y disparar `notifications:reminders`.

Qué hacer si falla:

- Verificar que el usuario haya activado el bot enviando "I allow callmebot to send me messages" a su número
- Logs: agregar `Log::warning()` en el servicio cuando falle

5.5 GOOGLE RECAPTCHA V3 (FASE 3 — PENDIENTE)

Propósito: Bloquear bots en el formulario de registro.

Credenciales:

- Site key y Secret key desde <https://www.google.com/recaptcha/admin>

Configuración en `.env`:

```
RECAPTCHA_SITE_KEY=[CONFIGURAR EN .env]
RECAPTCHA_SECRET_KEY=[CONFIGURAR EN .env]
```

Implementación pendiente:

1. En `register.blade.php` head:

```
<script src="https://www.google.com/recaptcha/api.js?render={{ env('RECAPTCHA_SITE_KEY') }}"></script>
```

2. En el form de registro, hidden input + script que ejecute `grecaptcha.execute()` antes del submit y rellene el hidden

3. En `RegisterController::store()`:

```
$response = Http::asForm()->post('https://www.google.com/recaptcha/api/siteverify', [
    'secret' => env('RECAPTCHA_SECRET_KEY'),
    'response' => $request->input('g-recaptcha-response'),
]);
if (!$response->json('success') || $response->json('score') < 0.5) {
    throw ValidationException::withMessages(['captcha' => 'Validación captcha fallida']);
}
```

Cómo probar: usar <https://www.google.com/recaptcha/api2/demo> o el dashboard que muestra eventos en vivo.

6. INFRAESTRUCTURA Y HOSTING

6.1 DATOS DEL SERVIDOR HOSTINGER PREMIUM

ATRIBUTO	VALOR
Proveedor	Hostinger
Plan	Premium Web Hosting
Panel	hPanel (https://hpanel.hostinger.com)
Sistema operativo	Linux compartido
PHP version	8.3 (seleccionable en hPanel → Avanzado → Configuración PHP)
MySQL version	8.x
Web server	Apache (front) + PHP-FPM (backend)
SSL	Let's Encrypt gratuito
SSH	Habilitable en hPanel · puerto 65002
Dominio	soypachonmundial.online
Email	Plan incluye hasta 100 cuentas

6.2 ESTRUCTURA DE CARPETAS EN EL SERVIDOR

Asumir usuario SSH `u123XXXXXX` (reemplazar con valor real de hPanel).

```

/home/u123XXXXXX/
??? public_html/           ? Symlink ? /home/u123XXXXXX/soypachonmundial/public
??? soypachonmundial/      ? Repo clonado desde GitHub
?   ??? app/
?   ??? bootstrap/
?   ??? config/
?   ??? database/
?   ??? public/           ? DocumentRoot real del dominio
?   ?   ??? index.php
?   ?   ??? build/       ? Assets compilados (commiteados)
?   ?   ??? ...
?   ??? resources/
?   ??? routes/
?   ??? storage/
?   ?   ??? app/
?   ?   ??? framework/
?   ?   ?   ??? cache/
?   ?   ?   ??? sessions/
?   ?   ?   ??? views/
?   ?   ??? logs/
?   ?       ??? laravel.log ? Archivo crítico para debugging
?   ??? vendor/           ? composer install
?   ??? tests/
?   ??? .env              ? Variables sensibles (chmod 600)
?   ??? artisan
??? backups/              ? Convención: backups de BD

```

Tamaños esperables:

- `vendor/` ~120 MB
- `storage/logs/laravel.log` crece según tráfico (rotar mensualmente)
- BD MySQL en hPanel: ~5-50 MB en producción típica
- Espacio total del plan: revisar quota en hPanel → **Inicio** → **Detalles del plan**

6.3 CONFIGURACIÓN DE PHP 8.3 RELEVANTE

En **hPanel** → **Avanzado** → **Configuración PHP**:

Extensiones requeridas (verificar que estén activas):

- `bcmath`, `ctype`, `curl`, `fileinfo`, `json`, `mbstring`
- `openssl`, `pdo_mysql`, `tokenizer`, `xml`, `zip`

Opciones recomendadas:

- `memory_limit` : 512M (default suele bastar; subir si dompdf falla)
- `max_execution_time` : 60 (suficiente para generar PDF de auditoría)
- `upload_max_filesize` : 10M
- `post_max_size` : 12M

6.4 CRON JOB EN HPANEL

Ubicación: hPanel → Avanzado → Cron Jobs → Crear nuevo Cron Job

Frecuencia: Cada minuto (* * * * *)

Comando exacto:

```
cd /home/u123XXXXXX/soypachonmundial && /usr/bin/php artisan schedule:run >> /dev/null 2>&1
```

Reemplazar `u123XXXXXX` con el usuario real. Si la ruta de PHP es distinta a `/usr/bin/php` , verificar con `which php` por SSH.

Verificación de que está corriendo:

```
tail -f ~/soypachonmundial/storage/logs/laravel.log
# Esperar ~1 minuto. Debería aparecer actividad si hay schedules pendientes.
```

6.5 SSL Y HTTPS

Activación:

- 1. hPanel → Avanzado → SSL
- 2. Buscar `soypachonmundial.online` en la lista
- 3. Click "Instalar SSL" (gratuito Let's Encrypt)
- 4. Esperar ~5 minutos hasta que aparezca "Activo" en verde
- 5. En la misma pantalla: activar **Forzar HTTPS**

Renovación: Let's Encrypt renueva automáticamente cada 90 días. Hostinger lo gestiona, no requiere intervención.

Verificación:

```
curl -sI https://soypachonmundial.online | head -5
# Debe responder 200 OK con cert válido

curl -sI http://soypachonmundial.online | grep -i location
# Debe redirigir a https://
```

6.6 VARIABLES DE ENTORNO DEL `.ENV` DE PRODUCCIÓN

Lista completa con descripción. Para valores sensibles usar `[CONFIGURAR EN .env]` .

```

# ??? App ???
APP_NAME="@SoyPachon"           # Mostrado en <title> del browser
APP_ENV=production              # Determina debug, log level, etc.
APP_KEY=[CONFIGURAR EN .env]    # base64:... - generar con php artisan key:generate
APP_DEBUG=false                 # En producción NUNCA true (expone trazas)
APP_TIMEZONE=America/Bogota     # Para todas las fechas
APP_URL=https://soypachonmundial.online

APP_LOCALE=es
APP_FALLBACK_LOCALE=es
APP_FAKER_LOCALE=es_ES

APP_MAINTENANCE_DRIVER=file
APP_MAINTENANCE_STORE=database

BCRYPT_ROUNDS=12                # Costo del hash de passwords

# ??? Logging ???
LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=error                  # En prod: error. En dev: debug.

# ??? Base de datos MySQL Hostinger ???
DB_CONNECTION=mysql
DB_HOST=localhost               # Mismo nodo en Hostinger
DB_PORT=3306
DB_DATABASE=[CONFIGURAR EN .env] # u123XXXXXX_pachon
DB_USERNAME=[CONFIGURAR EN .env] # u123XXXXXX_pachon_user
DB_PASSWORD=[CONFIGURAR EN .env]

# ??? Sesiones / colas / cache ???
SESSION_DRIVER=database
SESSION_LIFETIME=120            # Minutos
SESSION_ENCRYPT=false
SESSION_PATH=/
SESSION_DOMAIN=null
SESSION_SECURE_COOKIE=true      # Solo cookies por HTTPS

BROADCAST_CONNECTION=log
FILESYSTEM_DISK=local
QUEUE_CONNECTION=database
CACHE_STORE=database
CACHE_PREFIX=

# ??? Email SMTP Hostinger ???
MAIL_MAILER=smtp
MAIL_HOST=smtp.hostinger.com
MAIL_PORT=465
MAIL_USERNAME=notificaciones@soypachonmundial.online
MAIL_PASSWORD=[CONFIGURAR EN .env]
MAIL_ENCRYPTION=ssl
MAIL_FROM_ADDRESS="notificaciones@soypachonmundial.online"
MAIL_FROM_NAME="${APP_NAME}"

# ??? Vite ???
VITE_APP_NAME="${APP_NAME}"

```

6.7 PROCESO COMPLETO DE DEPLOY

Pre-flight local (en máquina del desarrollador):

```
$env:Path = "C:\laragon\bin\php\php-8.3.30-win32-vs16-x64;C:\laragon\bin\composer;$env:Path"
cd C:\laragon\www\soyPachonMundial

git status                # debe decir clean
php artisan test           # 86 passing
npm run build              # genera public/build/

git add -A
git commit -m "..."
git push origin master
```

En servidor Hostinger (SSH):

```
ssh -p 65002 u123XXXXXX@<host>
cd ~/soypachonmundial

git pull origin master      # trae código + assets compilados

composer install --no-dev --optimize-autoloader

php artisan migrate --force  # si hay migraciones nuevas

php artisan optimize:clear   # limpia caches viejos
php artisan config:cache     # cachea config
php artisan route:cache      # cachea rutas
php artisan view:cache       # precompila views
```

Verificación post-deploy:

```
curl -sI https://soypachonmundial.online | head -3
# Esperado: HTTP/2 200
```

En browser: **Ctrl+Shift+R** para forzar refresh del CSS.

6.8 PROCESO DE ROLLBACK

Si un deploy rompe producción:

Rollback rápido (15 segundos):

```
ssh -p 65002 u123XXXXXX@<host>
cd ~/soypachonmundial

git log --oneline -10        # identifica el último commit estable
git reset --hard <hash-anterior> # vuelve a ese commit

php artisan optimize:clear
php artisan config:cache
php artisan view:cache

# Si había migraciones nuevas que se aplicaron:
php artisan migrate:rollback --step=1 # rollea la última
```

Verificación:

```
curl -sI https://soypachonmundial.online | head -3
```

Comunicación durante rollback: activar modo mantenimiento si es público:

```
php artisan down --refresh=60 --secret="codigo-secreto-admin"
# Ahora todos ven "503 Service Unavailable" excepto quien pase ?secret=codigo-...
# Después del fix:
php artisan up
```

7. GUÍA DE OPERACIÓN Y ADMINISTRACIÓN

Para todo lo que sigue, el admin debe loguearse con `admin@soypachonmundial.com` (cambiar contraseña en primer login).

7.1 INGRESAR RESULTADOS OFICIALES DE PARTIDOS

1. Ir a `/admin/resultados`
2. En la sección "Pendientes de resultado" aparecen los partidos cuya hora ya pasó
3. En el partido correspondiente, ingresar `home_score` y `away_score`
4. Click "Guardar y calcular"
5. Aparece un flash message con la distribución de puntos
6. El ranking se actualiza automáticamente

Si te equivocás:

- En la sección "Últimos finalizados" aparece el partido
- Modificar los scores
- Click "Recalcular"
- Se sobrescriben los puntos y el ranking se rehace

7.2 GENERAR Y GESTIONAR CÓDIGOS DE INVITACIÓN

1. Ir a `/admin/codigos`
2. Stats arriba: cuántos hay disponibles / usados / desactivados
3. En el toolbar, indicar cuántos códigos generar (1-100) y click "+ Generar SPM-XXXX"
4. La tabla se actualiza con los códigos nuevos
5. Para enviar a los usuarios:
 - Click "[icon] Exportar disponibles" → abre modal con todos los códigos
 - "Copiar al portapapeles" → ya tenés la lista
 - Pegar en WhatsApp o email
6. Para desactivar un código (e.g. ya no se necesita): botón "Desactivar" en la fila

Buenas prácticas:

- Generar lotes de 10-20 por vez (no todos al inicio)
- Asignar un código a cada nuevo participante después de verificar su pago
- Llevar registro externo (Excel/Sheets) de quién recibió qué código

7.3 ACTUALIZAR EQUIPOS DE ELIMINATORIA

Cuando termina la fase de grupos y se conocen los clasificados:

1. Ir a `/admin/fixture`
2. Scrollar a "Dieciseisavos de Final"
3. En el partido a actualizar (e.g. #73 Clasificado A1 vs Clasificado B2): click "Editar"
4. Reemplazar:
 - `home_team` : "Clasificado A1" → "Colombia"
 - `away_team` : "Clasificado B2" → "Inglaterra"
 - `home_flag` : pegar emoji 🇨🇴
 - `away_flag` : pegar emoji 🇬🇧
 - `match_date` y `match_time` : ajustar si FIFA cambió el horario
 - `venue` : ingresar el estadio
5. Guardar
6. Repetir para los 16 partidos de dieciseisavos
7. Después de octavos: actualizar los 8 de cuartos
8. Y así sucesivamente

El `lock_datetime` se recalcula automático (`match_datetime` - 5 min).

7.4 RECALCULAR PUNTOS SI SE CORRIGE UN RESULTADO

Ver sección 7.1 — el botón "Recalcular" en `/admin/resultados` lo hace todo. Por línea de comandos:

```
php artisan predictions:calculate {match_id}
```

7.5 CONFIGURAR EL ACUMULADO

1. Ir a `/admin/configuracion`
2. En "Acumulado total (COP)" ingresar el valor (e.g. `600000` para 20 participantes × \$30.000)
3. Guardar
4. Verificar en `/ranking` : el desglose 60/25/15 ya muestra los valores correctos
5. En `/como-funciona#premio` la calculadora también respeta este valor por defecto

7.6 EXPORTAR REPORTE DE AUDITORÍA

1. Ir a `/auditoria/exportar` (también accesible desde el navbar como "↓ Auditoría")
2. Stats muestran cuántos registros tiene el reporte y la fecha de generación
3. Click "Descargar CSV" para abrir en Excel/Sheets
4. Click "Descargar PDF" para versión formal landscape

Cuándo exportar:

- Después de cada partido para tener trazabilidad
- Antes de anunciar el ganador para evidencia oficial
- Si un usuario reclama, mandarle el PDF del torneo completo

7.7 ACTIVAR/DESACTIVAR USUARIOS

1. Ir a `/admin/usuarios`
2. Buscar por nombre o email
3. En la fila del usuario, click "Desactivar" (en rojo) o "Activar" (en verde)
4. Cambio inmediato — el usuario perderá acceso al ranking y a `/predictions`

Casos típicos:

- Usuario reportó duplicado → desactivar el más nuevo
- Usuario reportó que su pago no se verificó → activar manualmente
- Limpieza pre-lanzamiento real → desactivar `user1..5@test.com`

7.8 CONFIGURAR EL VIDEO EXPLICATIVO

1. Subir video a YouTube (puede ser unlisted)
2. Copiar URL del video (formato watch,youtu.be o embed — los 3 funcionan)
3. Ir a `/admin/configuracion`
4. Pegar en "URL del video (YouTube)"
5. Guardar
6. Verificar en `/como-funciona` — el iframe debe cargar el video

7.9 DATOS DE DEMOSTRACIÓN

Cargar demo (para presentaciones o videos publicitarios):

```
php artisan db:seed --class=DemoSeeder
```

Produce: 10 usuarios @demo.com, 15 partidos finished, ranking poblado, acumulado \$500.000.

Limpiar después de la demo: Como no hay comando dedicado `demo:clean`, ver opciones en la sección 11.

8. COMANDOS ARTISAN DE REFERENCIA

COMANDO	DESCRIPCIÓN	CUÁNDO USARLO
<code>php artisan serve</code>	Servidor de desarrollo en http://127.0.0.1:8000	Desarrollo local
<code>php artisan test</code>	Corre los 86 tests	Antes de cada commit
<code>php artisan test --filter=X</code>	Solo tests que matchean X	Debug puntual
<code>php artisan migrate</code>	Aplica migraciones pendientes	Después de un git pull
<code>php artisan migrate --force</code>	Idem sin pedir confirmación	Producción
<code>php artisan migrate:fresh --seed</code>	Borra BD y re-corre todo (peligroso)	Solo dev local
<code>php artisan migrate:rollback --step=1</code>	Reversa la última migración	Rollback dev
<code>php artisan migrate:status</code>	Lista migraciones aplicadas/pendientes	Diagnóstico
<code>php artisan db:seed</code>	Corre <code>DatabaseSeeder</code> (admin + códigos + fixture)	Setup inicial
<code>php artisan db:seed --class=DemoSeeder</code>	Carga 10 usuarios demo + 15 matches calculados	Pre-grabar videos
<code>php artisan predictions:lock</code>	Bloquea pronósticos vencidos	Automático cada minuto
<code>php artisan predictions:calculate {id}</code>	Calcula puntos de un partido y recalcula ranking	Cuando se ingresa resultado
<code>php artisan notifications:reminders</code>	Envía emails de recordatorio	Automático cada minuto
<code>php artisan schedule:run</code>	Ejecuta una pasada del scheduler	Lo invoca el cron de Hostinger
<code>php artisan schedule:work</code>	Mantiene el scheduler corriendo en foreground	Útil en dev sin cron
<code>php artisan key:generate</code>	Genera nuevo <code>APP_KEY</code>	Setup inicial
<code>php artisan key:generate --show</code>	Lo muestra sin escribirlo al .env	Pre-deploy
<code>php artisan storage:link</code>	Crea symlink public/storage → storage/app/public	Una vez por servidor
<code>php artisan optimize:clear</code>	Limpia todos los caches	Después de cambiar .env, rutas, vistas
<code>php artisan config:cache</code>	Cachea config	Producción
<code>php artisan route:cache</code>	Cachea rutas	Producción
<code>php artisan view:cache</code>	Pre-compila vistas	Producción
<code>php artisan view:clear</code>	Solo limpia views compiladas	Debug de Blade
<code>php artisan tinker</code>	REPL interactiva con la app cargada	Diagnóstico, ejecución manual
<code>php artisan route:list</code>	Lista todas las rutas	Diagnóstico
<code>php artisan down</code>	Activa modo mantenimiento	Antes de deploy crítico
<code>php artisan down --refresh=60 --secret=xxx</code>	Mantenimiento con bypass	Idem con backdoor

COMANDO	DESCRIPCIÓN	CUÁNDO USARLO
<code>php artisan up</code>	Desactiva mantenimiento	Tras el deploy

9. SOPORTE PREVENTIVO

9.1 CHECKLIST DE VERIFICACIÓN SEMANAL

ITEM	CÓMO VERIFICAR	FRECUENCIA
Cron job activo	<code>tail -50 ~/soypachonmundial/storage/logs/laravel.log</code> debe mostrar actividad reciente	Semanal
SSL vigente	https://www.ssllabs.com/ssltest/analyze.html?d=soypachonmundial.online	Mensual
Espacio en disco	hPanel → Inicio → Detalles del plan	Semanal
Tamaño del log	<code>du -h ~/soypachonmundial/storage/logs/laravel.log</code> — rotar si > 500 MB	Semanal
Backup de BD	Listar <code>~/backups/</code> o revisar hPanel → Copias de seguridad	Semanal
Errores en log	<code>grep -i "error exception" storage/logs/laravel.log tail -50</code>	Diaria
Próximos partidos sin resultado	Query SQL de sección 3.6 #3	Diaria si Mundial activo
Códigos disponibles	<code>/admin/codigos</code> — generar más si quedan pocos	Semanal

9.2 CÓMO LEER LOGS DE LARAVEL

Ubicación: `storage/logs/laravel.log`

Formato típico:

```
[2026-06-15 14:23:51] production.ERROR: SQLSTATE[42000]:  
Syntax error or access violation: 1064 ... {"exception":"[object] ..."}  

```

Componentes:

- `[2026-06-15 14:23:51]` — timestamp UTC
- `production` — entorno (APP_ENV)
- `ERROR` — severidad: DEBUG, INFO, NOTICE, WARNING, ERROR, CRITICAL, ALERT, EMERGENCY
- mensaje + context JSON

Comandos útiles:

```
# Últimas 100 líneas
tail -100 ~/soypachonmundial/storage/logs/laravel.log

# Seguir en vivo
tail -f ~/soypachonmundial/storage/logs/laravel.log

# Errores hoy
grep "$(date +%Y-%m-%d)" ~/soypachonmundial/storage/logs/laravel.log | grep ERROR

# Buscar excepción específica
grep -A 5 "QueryException" ~/soypachonmundial/storage/logs/laravel.log

# Limpiar log (después de archivar)
> ~/soypachonmundial/storage/logs/laravel.log
```

Rotación recomendada: mensual. Mover y comprimir:

```
gzip -c storage/logs/laravel.log > storage/logs/laravel-$(date +%Y%m).log.gz
> storage/logs/laravel.log
```

9.3 ERRORES COMUNES Y SOLUCIÓN

Pronósticos que no se bloquean automáticamente:

Causa: cron de Hostinger no está corriendo o `predictions:lock` falla silencioso.

Diagnóstico:

```
# Por SSH
php artisan predictions:lock
# Debería responder: "Partidos pasados a 'live': X / Pronósticos bloqueados: Y"
```

Si no responde nada o tira error:

- Verificar en hPanel → Cron Jobs → ver "Última ejecución"
- Probar manualmente: `cd ~/soypachonmundial && php artisan schedule:run`
- Revisar `storage/logs/laravel.log` por errores

Solución temporal: correr manualmente cada cierto tiempo hasta arreglar el cron.

Puntos que no se calculan:

Causa típica: admin marcó el partido finished sin invocar el calculator.

Diagnóstico:

```
SELECT id, match_number, home_score_official, away_score_official, status
FROM matches WHERE status='finished' AND home_score_official IS NOT NULL;
-- Verificar que el partido en cuestión esté en este listado
```

Solución:

```
php artisan predictions:calculate {match_id}
```

Emails que no llegan:

Diagnóstico:

```
php artisan tinker
>>> Mail::raw('test', fn($m) => $m->to('vos@email.com')->subject('Prueba'));
```

Si tira error de auth: revisar `MAIL_USERNAME` y `MAIL_PASSWORD` en `.env`. Si tira timeout: probar otro puerto (587 + TLS). Si no tira error pero no llega: revisar spam, validar SPF/DKIM en hPanel → Emails.

Error 500 en producción:

Diagnóstico inmediato:

```
tail -50 ~/soypachonmundial/storage/logs/laravel.log | grep -A 10 ERROR
```

Causas comunes:

- `APP_KEY` vacío o inválido → `php artisan key:generate`
- Permisos de storage → `chmod -R 775 storage bootstrap/cache`
- `.env` corrupto → comparar contra `.env.production.example`
- Migraciones desincronizadas → `php artisan migrate --force`

Problemas de permisos en `storage` y `bootstrap/cache` :

```
chmod -R 775 storage bootstrap/cache
# Si el usuario web es distinto al SSH:
chown -R u123XXXXXX:u123XXXXXX storage bootstrap/cache
```

9.4 VERIFICAR QUE EL CRON JOB ESTÁ CORRIENDO

En hPanel:

1. Avanzado → Cron Jobs
2. Ver "Última ejecución" del job de `schedule:run`

Por SSH:

```
# El log debería tener entradas frescas (< 5 min)
ls -la ~/soypachonmundial/storage/logs/laravel.log

# Forzar manualmente para ver si funciona
cd ~/soypachonmundial && php artisan schedule:run
```

Si el comando manual funciona pero el cron no: problema en hPanel. Recrear el cron job.

9.5 MONITOREO DE LA BD

Tablas que crecen rápido:

- `sessions` — depende del tráfico. Limpiar sesiones viejas:

```
DELETE FROM sessions WHERE last_activity < UNIX_TIMESTAMP() - 7*24*3600;
```

- `cache` — debería rotar solo. Si crece mucho: `php artisan cache:clear`
- `match_notifications` — crece linealmente con # partidos × # users. No tocar.

Queries lentas: Activar `slow_query_log` en MySQL no es posible en Hostinger compartido. Alternativa: Laravel telemetry vía middleware.

Para diagnóstico puntual:

```
php artisan tinker
>>> DB::enableQueryLog();
>>> // ejecutar la acción problemática
>>> DB::getQueryLog();
// Verás cada query con su tiempo
```

10. SOPORTE CORRECTIVO

10.1 IDENTIFICAR Y REPRODUCIR UN BUG

Proceso recomendado:

1. Recolectar info del usuario:

- URL exacta donde aparece
- Captura de pantalla
- Email del usuario (para reproducir con su cuenta)
- Acción exacta que disparó el error
- Browser y dispositivo

2. Reproducir en local:

```
# En local, crear un usuario con los mismos datos:
php artisan tinker
>>> User::factory()->create(['email' => $userEmail, 'is_active' => true]);
```

3. Reproducir el escenario:

- Cargar mismos datos en BD local
- Hacer la misma acción
- Si reproduce: tener el bug en la mano

4. Si no reproduce en local: descargar el log de producción y buscar el error:

```
# Por SSH
grep "$(date +%Y-%m-%d)" storage/logs/laravel.log | grep ERROR | head -20
```

5. **Una vez identificado:** corregir + escribir test que falla con el bug y pasa con el fix.

10.2 HOTFIX URGENTE EN PRODUCCIÓN

Si la app está rota y hay usuarios afectados:

```
# 1. Activar mantenimiento con bypass para vos
ssh -p 65002 u123XXXXXX@<host>
cd ~/soypachonmundial
php artisan down --refresh=60 --secret="hotfix-2026"

# 2. Aplicar el fix (1 archivo, mínimo cambio)
nano app/Http/Controllers/XController.php # o lo que sea
# editar...

# 3. Limpiar cache de rutas/vistas/config
php artisan optimize:clear

# 4. Verificar el fix accediendo con ?secret=hotfix-2026 al URL afectado
# desde tu browser

# 5. Cuando confirmes que funciona:
php artisan up

# 6. IMPORTANTE: commitear el hotfix
git add app/Http/Controllers/XController.php
git config --local user.name "...
git config --local user.email "...
git commit -m "hotfix: descripción breve"
git push origin master
```

Después en local: `git pull` para no perder el hotfix.

10.3 ROLLBACK COMPLETO

Ver sección 6.8.

10.4 RESTAURAR BD DESDE BACKUP

```
ssh -p 65002 u123XXXXXX@<host>

# Backup del estado actual antes de restaurar (por si algo sale mal)
mysqldump -u u123XXXXXX_pachon_user -p u123XXXXXX_pachon \
  > ~/backups/pre-restore-$(date +%F-%H%M).sql

# Restaurar (ajustar path al backup deseado)
gunzip -c ~/backups/spm-2026-06-15.sql.gz | mysql -u u123XXXXXX_pachon_user -p u123XXXXXX_pachon

# Verificar
mysql -u u123XXXXXX_pachon_user -p u123XXXXXX_pachon \
  -e "SELECT COUNT(*) FROM users; SELECT COUNT(*) FROM predictions;"
```

10.5 DEPURAR UN ERROR ESPECÍFICO

Mostrar el error completo (solo en local con `APP_DEBUG=true`):

- Abrir la URL → Laravel muestra Ignition (página de error visual)

En producción (`APP_DEBUG=false`):

- Usuario solo ve "500 Server Error"
- Detalle en `storage/logs/laravel.log`

Depuración interactiva:

```
php artisan tinker
>>> $user = User::where('email', 'carlos@demo.com')->first();
>>> $user->predictions->count();
>>> // explorar como en un REPL
```

Dump en código (no usar en producción más de minutos):

```
\Log::info('Estado debug', ['user' => $user, 'data' => $data]);
// Reflejar en log y revisar con tail -f
```

10.6 ARCHIVOS CRÍTICOS — NO MODIFICAR SIN PRUEBA PREVIA

ARCHIVO	RAZÓN
<code>app/Services/PredictionScoringService.php</code>	Cambia los puntos del torneo. Cubierto por 20 tests parametrizados.
<code>app/Services/RankingService.php</code>	Lógica de desempate. Afecta el ganador del torneo.
<code>app/Console/Commands/LockPredictions.php</code>	Garantiza el cierre justo a tiempo.
<code>app/Console/Commands/CalculatePredictions.php</code>	Recalcula todos los puntos.
<code>app/Http/Middleware/EnsureActive.php</code> y <code>EnsureAdmin.php</code>	Seguridad de acceso.
<code>routes/web.php</code>	Cualquier cambio puede romper navegación.
Migraciones aplicadas	NUNCA editar una migración que ya corrió en producción — crear una nueva

Regla general: todo cambio en estos archivos debe:

1. Tener test asociado
2. Pasar `php artisan test` antes del commit
3. Probarse en local con la BD de demo (`DemoSeeder`)
4. Probarse en staging si existe
5. Desplegarse en horario de bajo tráfico

11. SOPORTE EVOLUTIVO

11.1 AGREGAR UNA NUEVA VISTA

Ejemplo: vista pública `/blog` con un listado.

```
# 1. Definir ruta en routes/web.php
Route::view('/blog', 'blog.index')->name('blog.index');
# o si necesita controller:
Route::get('/blog', [BlogController::class, 'index'])->name('blog.index');

# 2. Crear el controller (si aplica)
php artisan make:controller BlogController

# 3. Crear la vista
# resources/views/blog/index.blade.php:
@extends('layouts.app')
@section('title', 'Blog')
@section('content')
    <div class="max-w-5xl mx-auto px-4 py-8">
        <h1 class="font-display font-bold text-display-l text-pitch uppercase">Blog</h1>
        {{-- contenido --}}
    </div>
@endsection

# 4. Agregar link en navbar si corresponde
# resources/views/components/nav.blade.php

# 5. Test
# tests/Feature/BlogTest.php:
public function test_blog_es_publico(): void {
    $this->get(route('blog.index'))->assertOk();
}

# 6. Build y commit
npm run build
git add -A && git commit -m "feat: vista pública /blog"
```

11.2 AGREGAR UNA NUEVA TABLA

Ejemplo: tabla `payments` para tracking de pagos.

```

# 1. Generar migración
php artisan make:migration create_payments_table

# 2. Editar database/migrations/...create_payments_table.php
Schema::create('payments', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained()->cascadeOnDelete();
    $table->integer('amount'); // en centavos
    $table->string('reference', 100)->unique();
    $table->enum('status', ['pending', 'verified', 'rejected']);
    $table->timestamp('verified_at')->nullable();
    $table->timestamps();
    $table->index('status');
});

# 3. Aplicar migración
php artisan migrate

# 4. Crear modelo
php artisan make:model Payment

# 5. En el modelo, definir fillable + relaciones + casts
# app/Models/Payment.php:
protected $fillable = ['user_id', 'amount', 'reference', 'status', 'verified_at'];
protected $casts = ['verified_at' => 'datetime'];

public function user(): BelongsTo { return $this->belongsTo(User::class); }

# 6. Agregar relación inversa en User si conviene
public function payments(): HasMany { return $this->hasMany(Payment::class); }

# 7. Si la tabla tiene tests, escribirlos
# 8. Commit

```

11.3 AGREGAR UN COMANDO ARTISAN AL SCHEDULER

```

# 1. Crear comando
php artisan make:command NombreComando

# 2. Editar app/Console/Commands/NombreComando.php
protected $signature = 'app:nombre-comando';
protected $description = 'Descripción breve';

public function handle(): int {
    // lógica aquí
    $this->info('OK');
    return self::SUCCESS;
}

# 3. Agregar al scheduler en routes/console.php
Schedule::command('app:nombre-comando')
    ->everyTenMinutes()
    ->withoutOverlapping();

# 4. Probar manualmente
php artisan app:nombre-comando

# 5. Verificar que el scheduler lo lista
php artisan schedule:list

```

11.4 AGREGAR UN COMPONENTE BLADE DEL DESIGN SYSTEM


```
# 1. Crear archivo en resources/views/components/
# Ejemplo: nuevo card de notificación
# resources/views/components/alert-card.blade.php:

@props([
    'variant' => 'info', // info, success, warning, danger
    'title' => null,
])

@php
$classes = match($variant) {
    'success' => 'bg-pitch-mist border-pitch text-pitch',
    'warning' => 'bg-gol/20 border-gol text-pitch-deep',
    'danger' => 'bg-alerta/10 border-alerta text-alerta',
    default => 'bg-bone-soft border-line text-ink',
};
@endphp

<div {{ $attributes->class(['border rounded-md p-4', $classes]) }}>
    @if($title)
        <p class="font-display font-bold text-display-s uppercase mb-2">{{ $title }}</p>
    @endif
    <div class="text-body-s">{{ $slot }}</div>
</div>

# 2. Usar en cualquier vista:
<x-alert-card variant="success" title="¡Listo!">
    Tu pago fue verificado correctamente.
</x-alert-card>
```

Mantener el design system:

- Usar SIEMPRE los tokens de paleta: `pitch`, `gol`, `bone`, `ink`, `line`, `alerta`
- Usar las fuentes: `font-display`, `font-body`, `font-mono`
- Para puntos, usar las **5 clases unificadas** documentadas en sección 6 de CLAUDE.md

11.5 IMPLEMENTAR INTEGRACIONES FASE 3

Ver sección 5 — cada subsección tiene pasos específicos:

- 5.3 Google OAuth
- 5.2 API-Football
- 5.4 CallMeBot WhatsApp
- 5.5 Google reCAPTCHA

11.6 CONVENCIONES DE CÓDIGO

Nombres:

- Clases: `PascalCase` (ej. `PredictionsCalculator`)
- Métodos y variables: `camelCase` (ej. `calculatePoints`)
- Constantes: `SCREAMING_SNAKE_CASE` (ej. `TYPE_REMINDER`)
- Tablas y columnas BD: `snake_case` (ej. `home_score_official`)

- Rutas URL: `kebab-case` en español si es público (ej. `/como funciona`), inglés si es API (ej. `/predictions`)
- Nombres de rutas Laravel: `dot.notation` (ej. `predictions.index` , `admin.codes.generate`)

Estructura de controllers:

- Un método por acción HTTP típica (`index` , `show` , `store` , `update` , `destroy`)
- Inyectar dependencias por constructor o parámetros de método
- Validación con `$request->validate([...])` al inicio del método
- Retornar `RedirectResponse` con flash en POST/PATCH, `View` en GET, `JsonResponse` en API

Uso de Services:

- Si la lógica se usa en más de un lugar (controller + comando, dos controllers) → Service
- Si tiene reglas de negocio complejas testeables → Service
- Si es CRUD simple → directamente en el controller

Commits:

- Formato: `tipo: descripción corta en español`
- Tipos: `feat` , `fix` , `design` , `build` , `release` , `deploy` , `refactor` , `docs`
- Co-author al final:

```
Co-Authored-By: Claude Opus 4.7 <noreply@anthropic.com>
```

Tests:

- `tests/Feature/` para tests HTTP / integración
- `tests/Unit/` para lógica pura (ej. `PredictionScoringServiceTest`)
- Usar `RefreshDatabase` en feature tests
- Usar `#[DataProvider]` para casos parametrizados

12. GLOSARIO

TÉRMINOS DE LARAVEL

TÉRMINO	DEFINICIÓN
Artisan	CLI de Laravel: <code>php artisan <comando></code> . Permite ejecutar tareas internas (migrate, tests, seed, comandos custom).
Blade	Motor de plantillas de Laravel. Archivos <code>.blade.php</code> . Soporta <code>@if</code> , <code>@foreach</code> , <code><x-componente></code> .
Component (x-)	Pieza reutilizable de Blade invocada con <code><x-nombre></code> . Vive en <code>resources/views/components/</code> .
Eloquent	ORM de Laravel. Cada Model representa una tabla y permite consultas tipo <code>User::where('email', ...)->first()</code> .
Facade	Acceso estático a servicios: <code>Auth::user()</code> , <code>DB::table('...')</code> , <code>Mail::raw(...)</code> .
Middleware	Capa que se ejecuta antes/después del controller. Ej: <code>auth</code> , <code>ensure.active</code> , <code>admin</code> .
Migration	Archivo PHP que define cambios al esquema de la BD. <code>php artisan migrate</code> los aplica.
Model	Clase Eloquent que representa una tabla. Ej: <code>User</code> , <code>Game</code> , <code>Prediction</code> .
Notification	Mensaje enviado por mail/SMS/etc. Clase en <code>app/Notifications/</code> . Se dispara con <code>\$user->notify(new ...)</code> .
Route	URL + método HTTP + acción. Definidas en <code>routes/web.php</code> .
Scheduler	Sistema de cron interno de Laravel. Se configura en <code>routes/console.php</code> . Se invoca cada minuto vía <code>php artisan schedule:run</code> .
Seeder	Archivo PHP que llena la BD con datos. Ej: <code>MatchSeeder</code> carga los 72 partidos. <code>php artisan db:seed</code> los corre.
Service	Clase con lógica de negocio reutilizable. Vive en <code>app/Services/</code> .
Tinker	REPL interactiva: <code>php artisan tinker</code> . Permite ejecutar código PHP con la app cargada.

TÉRMINOS DEL NEGOCIO

TÉRMINO	DEFINICIÓN
Acumulado	Suma total recaudada de todos los cupos pagados. Distribuido 60/25/15 al cierre del torneo.
Activo / Inactivo	Atributo del usuario (<code>users.is_active</code>). Solo activos ven <code>/predictions</code> y <code>/ranking</code> . Se vuelve activo al usar un código válido.
Auditoría	Transparencia total: cualquier usuario puede ver los pronósticos de cualquier otro en partidos ya finalizados. Disponible en <code>/ranking/u/{id}</code> y exportable como CSV/PDF.
Bloqueo	Cierre automático del pronóstico 5 min antes del inicio del partido. Determinado por <code>matches.lock_datetime</code> . Una vez bloqueado, el campo del marcador es read-only.
Código de invitación / activación	String tipo <code>SPM-XXXX</code> (4 caracteres alfanuméricos sin 0/O/I/1/L). Generado por el admin, se entrega al usuario tras verificar pago. Cada código es de un solo uso.
Cupo	Pago único para participar (actualmente \$30.000 COP). Todo lo recaudado va al acumulado, distribuido entre el top 3.
Cuartos de final	Fase de eliminatoria con 8 equipos (4 partidos). En el código <code>phase='cuartos'</code> .
Dieciseisavos	Primera fase de eliminatoria con 32 equipos (16 partidos). <code>phase='dieciseisavos'</code> .
Eliminatoria	Fases post-grupos: dieciseisavos → octavos → cuartos → semifinal → 3er_puesto + final.
Exacto	Pronóstico que sumó 5 puntos (ambos marcadores correctos). Usado como tiebreak en el ranking.
Fase de Grupos	Primera etapa del Mundial. 12 grupos (A..L) de 4 equipos cada uno. 3 jornadas × 24 partidos × 3 = 72 partidos.
Fixture	Calendario completo de partidos. En este proyecto: 104 partidos (72 grupos + 32 eliminatoria).
Octavos de final	Fase con 16 equipos (8 partidos). <code>phase='octavos'</code> . Coloquialmente la gente puede decir "octavos" para referirse a la ronda de 16; aquí seguimos la convención FIFA.
Pronóstico	Marcador (<code>home_score</code> , <code>away_score</code>) que el usuario predice para un partido. Almacenado en <code>predictions</code> .
Ranking	Tabla ordenada de participantes por puntos descendente, desempate por exactos. Cache en tabla <code>rankings</code> .
Semifinales	Fase con 4 equipos (2 partidos). <code>phase='semifinal'</code> .
Tiempo reglamentario	Los 90 minutos + tiempo añadido. Excluye prórroga y penales. Solo el resultado del tiempo reglamentario cuenta para puntos.

ACRÓNIMOS

ACRÓNIMO	SIGNIFICADO
API	Application Programming Interface
BD	Base de datos
CSRF	Cross-Site Request Forgery — token de seguridad incluido en forms
CSV	Comma-Separated Values
CTA	Call To Action — botón principal de una sección
DOM	Document Object Model
ERD	Entity-Relationship Diagram
FIFA	Fédération Internationale de Football Association
FK	Foreign Key — clave foránea
HTTPS	HyperText Transfer Protocol Secure
MVC	Model-View-Controller — patrón arquitectónico
PDF	Portable Document Format
PK	Primary Key — clave primaria
PHP	PHP: Hypertext Preprocessor
REPL	Read-Eval-Print Loop
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
SSH	Secure Shell
SSL	Secure Sockets Layer (sucesor: TLS)
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
UX	User Experience
YYMMDD	Formato de fecha año-mes-día

13. HISTORIAL DE VERSIONES

VERSIÓN	FECHA	DESCRIPCIÓN	TAG GIT
v1.0	2026-05-26	MVP Fase 1 — Lanzamiento inicial en producción. 8 pasos del MVP completados: autenticación, fixture, pronósticos, motor de puntos, ranking, admin, notificaciones email, deploy a Hostinger.	v1.0.0 (c774a8f)
v1.1	2026-05-27	Design system aplicado completo + modal Ver Pronósticos + exportación CSV/PDF + página ¿Cómo funciona? + distribución de premios actualizada a 60/25/15.	master (último)

Plantilla para futuras entradas:

| v1.x | YYYY-MM-DD | Descripción de los cambios principales | vx.y.z (hash) |

CÓMO MANTENER ESTE DOCUMENTO

Este documento debe actualizarse cuando se hagan cambios significativos al proyecto. Para que no quede desactualizado:

CAMBIO	SECCIÓN(ES) A ACTUALIZAR
Nueva versión de Laravel/PHP/Node	Sección 1.2 (stack)
Nueva tabla en BD	Sección 3.1 (ERD), 3.2 (descripción), 3.4 (migraciones)
Nueva ruta o módulo	Sección 2.3 (estructura), 4 (módulos funcionales)
Nuevo comando Artisan	Sección 8 (tabla de comandos), sección 4 si pertenece a un módulo
Nueva integración externa	Sección 5 (APIs)
Cambio en .env variables	Sección 6.6 (variables de entorno)
Cambio en convenciones	Sección 11.6 (convenciones)
Nueva versión de la app	Sección 13 (historial)

Quién mantiene:

- El responsable técnico de cada release debe actualizar las secciones afectadas en el mismo PR del cambio funcional
- Una vez por trimestre: revisar el documento completo y limpiar referencias obsoletas

Generar el PDF actualizado:

```
php artisan docs:generate-pdf
# genera docs/informe_tecnico_soypachonmundial.pdf
```

Cómo actualizar la versión del documento:

- Cambiar la línea inicial: Versión del documento: 1.x
- Agregar entrada en sección 13 si corresponde a una versión nueva de la app
- Commit con mensaje: docs: actualizar informe técnico para v1.x

